CampusTrack – Centralized Lost and Found Platform for College Campuses

A PROJECT REPORT

Submitted By :

Suryakant Jaiswar

Seat No : 510512446

Bachelor in Computer Science (Third Year)

Under the Guidance of

RAGVINDER KAUR

(Assistant Professor)

Achievers College of Commerce and Management, Kalyan (W)

(Affiliated to University of Mumbai)

ACADEMIC YEAR: 2026–2027

DECLARATION

We hereby declare that the project entitled “CampusTrack: A Secured Centralized Lost and Found Platform for College Campuses” submitted by us in partial fulfilment of the requirements for the Third Year Bachelor of Science in Computer Science (T.Y.B.Sc. C.S.) is our original work carried out under the guidance of Ms. Ragvinder Kaur Chadha at Achievers College of Commerce and Management, Kalyan (W).

We further declare that this project has not been submitted, either in whole or in part, to any other university or institution for the award of any degree or diploma. The information and references used in this project have been duly acknowledged.

We take full responsibility for the authenticity and originality of the work presented in this project report.

Project Title: CampusTrack: A Secured Centralized Lost and Found Platform for College Campuses

Name of Student : Suryakant Jaiswar

Seat Number: 510512446

Class : T.Y.B.Sc. Computer Science

Academic Year: 2026–2027

Place: Kalyan

Date: 30/10/2026

Signature of Student :

_____ (Suryakant Jaiswar)

Project Guide:

Ms. Ragvinder Kaur Chadha

Signature of Project Guide: _____

TABLE OF CONTENTS

Chapter	Topic	Page No.
-	Official Declaration	ii
1	Introduction	1
2	Problem Definition	5
3	Objectives	8
4	Requirement Specification	11
5	Existing System	16
6	Proposed System	19
7	System Analysis & Design	25
8	Methodology	32
9	System Implementation & Source Code Listings	36
10	Testing & Verification	54
11	Results / Screenshots	57
12	Conclusion	62
13	Future Scope	64
14	References	66
App.	Technical Appendices	67

CHAPTER-1

1. INTRODUCTION

1.1 Background and Motivation

Tertiary educational institutions, colleges, and university campuses represent dynamic micro-ecosystems characterized by high population density, constant physical transit, and diverse infrastructure. On any given academic day, thousands of undergraduate students, postgraduate researchers, faculty members, lab assistants, and administrative staff move between multi-story academic wings, computerized laboratories, central libraries, cafeterias, auditoriums, and sports facilities.

During these routine transitions between lectures, examination halls, and research centers, personal belongings are inevitably forgotten, misplaced, or inadvertently exchanged. Misplaced items span a wide spectrum of utility and financial worth:

- Critical Academic Credentials: Official university identity cards, library barcoded passes, examination hall tickets, and syllabus laboratory journals.
- High-Value Electronic Assets: Scientific programmable calculators, laptop power chargers, wireless Bluetooth earbuds, portable SSDs, and pen drives containing semester source code.
- Personal Valuables: Wallets, government identity cards (Aadhaar cards, PAN cards, driving licenses), spectacles, motorcycle keys, and branded apparel.

When an asset such as a scientific calculator or university hall ticket is lost hours before a practical or terminal semester examination, the resulting anxiety and operational disruption are severe. The student is forced to abandon exam preparation to frantically tour campus buildings, interrogate security guards, and post fragmented pleas across informal messaging groups.

1.2 Domain Challenges in Traditional Campus Environments

Historically, higher education campuses manage lost and found belongings through decentralized, ad-hoc, and manual mechanisms:

- Paper Registers at Gatehouses: Security guards maintain handwritten registers where finders log turned-in items. These logbooks lack photographic records, provide zero search capabilities, and frequently suffer from illegible handwriting.
- Corridor Notice Boards: Physical paper notices pinned to corkboards in main corridors tear, get covered by club flyers, and fail to provide immediate notifications to owners.
- Informal Social Media Groups: Students frequently post about lost articles in unofficial WhatsApp or Telegram batch chats. However, these notices quickly get buried beneath hundreds of casual messages, expose personal mobile numbers to harassment, and provide zero verification mechanisms to stop dishonest individuals from claiming items that do not belong to them.

1.3 Institutional Context: Achievers College

The CampusTrack project was conceptualized and engineered specifically for Achievers College of Commerce and Management, Kalyan (W). Built using React 19, Express.js 5, Node.js, and an embedded SQLite database managed with Prisma ORM, CampusTrack unifies the entire lost item recovery lifecycle into an intuitive, fraud-resistant digital hub.

1.4 What is the Project About?

CampusTrack is an institutional lost-and-found management workstation designed for campus students, faculty members, and security personnel. Its purpose is to take uncertain, scattered reports of lost or found belongings and convert them into structured, verifiable, and actionable digital recovery records.

The application is intentionally positioned as a secure intermediary between informal student discovery and physical security desk verification. It introduces an Anti-Fraud Security Challenge Protocol, enabling finders to withhold concealed identifying characteristics as questions that only genuine owners can answer before any physical meeting occurs.

1.5 Why Was This Topic Selected?

The topic "CampusTrack" was chosen because the recovery of misplaced personal property is a high-frequency, tangible problem experienced by every college campus community. The project was selected with both practical institutional impact and rigorous educational software engineering objectives in mind.

- To Solve a Real-World Problem: Eliminates permanent property loss, fraudulent claims, privacy invasions, and security desk logbook clutter.
- To Build a Production-Grade Web Application: Uses React 19 Single Page Architecture, Express 5 REST APIs, Multer multipart streaming, and Prisma ORM relational persistence.

1.6 Importance of CampusTrack

CampusTrack transforms an informal, vulnerable activity into a structured, auditable recovery pipeline:

Source Incident (Lost/Found) → Application Posting (+ Photo & Question) → Public Catalog Indexing → Claim Verification (+ Proof Document) → Poster Review (Approve/Reject) → Safe Handover at Security Desk

1.7 Project at a Glance

Field	Value
Project title	CampusTrack – Centralized Lost and Found Platform for College Campuses
Project release	v1.0.0 – Production & Security Hardened Release
Application status	Deployed, fully operational, and tested end-to-end
Prepared by	Suryakant Jaiswar (Seat No: 510512446)
Project guide	Ms. Ragvinder Kaur Chadha (Assistant Professor)
Institution	Achievers College of Commerce and Management, Kalyan (W)
Affiliation	Affiliated to University of Mumbai
Academic Year	2026–2027
Repository	github.com/proot-distrologinubuntu275/Findit
Core Tech Stack	React 19, Tailwind CSS v4, Node.js, Express 5, Prisma ORM, SQLite

CHAPTER-2

2. PROBLEM DEFINITION

2.1 Problem Definition & Context

In collegiate environments, the recovery of misplaced personal property is hampered by a lack of central coordination, severe information fragmentation, and the absence of verification protocols. When an item is lost, the victim has no single authoritative location to query. Conversely, when an honest individual finds an article, they have no secure, privacy-preserving method to locate the legitimate owner.

1) Information Asymmetry & Fragmentation

In typical campus operations, reports of found belongings are scattered across disparate gatehouses, faculty rooms, and departmental offices. If a student leaves a calculator in the 3rd Floor Electronics Lab, a peon may deposit it with the department clerk, while the student searches the ground-floor security gate. Because these distinct physical points maintain isolated paper notebooks, the item remains unclaimed indefinitely.

2) False Claims and Opportunistic Fraud

When students resort to broadcasting lost or found announcements on public messaging channels (such as college WhatsApp batches or Instagram confession pages), unscrupulous individuals exploit the situation. For instance, when a found wireless earbud case is broadcast publicly, any member can privately message the finder claiming ownership. Because finders lack formal investigative guidelines, items are routinely handed over to the first claimant rather than the rightful owner.

2.2 Who Faces the Problem?

- Undergraduate & Postgraduate Students: Face severe financial and academic consequences when calculators, identity cards, or project laptops are lost.
- Teaching Faculty & Lab Staff: Regularly lose proprietary storage drives, lecture presentation notes, and specialized lab equipment during classroom rotations.
- Campus Security Staff: Suffer operational bottlenecks trying to catalog physical property lockers with inadequate, illegible paper ledgers.
- College Administration: Faces reputational damage and student grievances regarding perceived lack of campus safety and security.

2.3 Limitations of Current Manual Approaches

Approach	Operational Mechanism	Core Deficiencies & Failure Modes
Physical Security Registers	Handwritten logs at college gatehouses.	No keyword searchability; entries cannot be cross-referenced across gates; zero photo capture; logs degrade over time.
Corridor Notice Boards	Paper posters pinned to corridor corkboards.	Low visibility; flyers get torn or covered; static information cannot be updated once recovered; zero privacy protection.
Social Media Batches	Unmoderated WhatsApp, Telegram, or Instagram posts.	Notices vanish beneath chat spam; exposes personal phone numbers to unsolicited calls; vulnerable to opportunistic theft.

2.4 Why Is the Project Needed?

Campus communities urgently require an authoritative, structured, and institutional platform that guarantees:

- Instantaneous Campus-Wide Indexing: Allowing any student to search across all departments and locations within milliseconds.

- Verifiable Ownership Claims: Guaranteeing that claimants must prove ownership before receiving items.
- Student Data Privacy: Ensuring personal contact info is never publicly exposed to unauthorized users.
- Administrative Auditing: Giving college authorities visibility into lost item recovery metrics and dispute resolutions.

CHAPTER-3

3. OBJECTIVES

3.1 Project Objectives

- 1. Centralized Academic Hub: Build a responsive web application serving as the single point of truth for lost and found items across the entire college campus.
- 2. Dual Reporting Pipelines: Develop independent, intuitive workflows for submitting both "Lost Item" notifications and "Found Item" registrations.
- 3. Anti-Fraud Verification Protocol: Design an interactive challenge-question verification engine preventing dishonest claims of valuable electronics and personal effects.
- 4. Server-Side Multipart Media Handling: Implement a robust image processing pipeline with Multer to store and serve high-resolution item photographs on disk.
- 5. Stateless Security & Authorization: Implement JSON Web Token (JWT) authentication and Bcrypt password hashing to protect private user data and API routes.
- 6. Sub-Second Search & Multi-Filter Querying: Engineer high-speed client and server filtering across categories, campus buildings, and item status.
- 7. Interactive Claim Management: Allow item posters to review answers, inspect uploaded purchase bills or proof images, and approve or reject claims.
- 8. Campus Security Administration Console: Deliver a specialized administrative dashboard enabling security personnel to oversee all listings, resolve disputes, and delete spam.
- 9. Zero-Configuration Relational Persistence: Utilize SQLite 3 with Prisma ORM to ensure complete project portability across local development and hosting servers.
- 10. Production Governance & Versioning: Deliver a release (v1.0.0) hosted on GitHub with comprehensive documentation and automated runners.

3.2 Scope of the Project (In Scope)

Area	Included Capabilities
Frontend	Responsive React 19 SPA, Tailwind CSS v4 styling, Lucide icons, live search, multi-category filters, claim modals, and dashboard views.
Backend	Express 5 RESTful API, JWT bearer auth middleware, Multer disk storage, status state transitions, and notification dispatchers.
Database	SQLite relational storage (dev.db) managed via Prisma ORM with cascade constraints.
Security	Bcrypt password hashing (10 salt rounds), JWT token signing, CORS boundaries, input sanitization, and 5MB image upload limits.
Admin	Campus security oversight portal, student account directory, listing moderation, and analytical metric counters.

3.3 Outside the Current Release

- Facial recognition or biometric identification of claimants.
- Inter-university synchronization across different collegiate campuses.
- Automated drone delivery of recovered belongings.

3.4 Completion Criteria

Criterion	Completion Evidence
Dual reporting pipeline	Lost and Found application forms operational with photo uploads.
Anti-fraud verification	Challenge question verification functional; posters can approve/reject claims.
Database persistence	Relational tables (User, Item, Claim, Notification) verified in SQLite.
Security & Auth	JWT authentication guards active; password hashes stored securely.
Campus Admin Console	Administrative metrics and student moderation portal fully functional.

CHAPTER-4

4. REQUIREMENT SPECIFICATION

4.1 Hardware Requirements

Hardware Component	Minimum Operational Specification	Recommended Production Specification
Client Processor	Dual-Core 1.6 GHz x86/ARM Processor	Quad-Core 2.4 GHz Intel Core i5 / Apple M-Series
Client Memory (RAM)	2 GB System Memory	4 GB to 8 GB System Memory
Client Display	360x640 Mobile Touch Display	1920x1080 Full HD Display
Server Hardware	1 vCPU, 1 GB RAM, 10 GB Disk (VPS or Local PC)	2 vCPU, 4 GB RAM, 40 GB SSD Storage
Network Connection	Standard Campus Wi-Fi / 4G Cellular (1 Mbps)	High-Speed Campus LAN / 5G Connection (10+ Mbps)

4.2 Software Requirements

Layer	Software Component	Version & Architectural Role
Runtime	Node.js	v18.0.0+ LTS (JavaScript Server Execution Engine)
Backend Framework	Express.js	v5.3.0 (HTTP RESTful Microservices & Middleware)
Database Engine	SQLite 3	Embedded Relational ACID-Compliant Database File
ORM	Prisma ORM	v6.4.1 (Schema Modeling, Migrations, Client Engine)
Media Handler	Multer	v2.5.0 (Multipart/Form-Data Image File Streaming)
Security Tokens	JSON Web Token (jsonwebtoken)	v9.0.3 (HMAC SHA-256 Stateless Bearer Tokens)
Cryptography	BcryptJS	v3.0.3 (Adaptive Salting & Password Hashing)
Frontend Library	React	v19.0.0 (Declarative UI Component Architecture)
Build Bundler	Vite	v8.0.0 (Fast HMR & Optimized Production Build Tool)
Styling Framework	Tailwind CSS	v4.0.0 (Utility-First Responsive Design Engine)
Icons	Lucide React	v0.400+ (Lightweight Vector SVG Icon Library)
Version Control	Git & GitHub	Distributed Versioning & GitHub Cloud Repository

4.3 Functional Requirements

Req. ID	Feature Scope	Requirement Description & System Behavior
FR-01	Student Registration	The system shall register new users with name, valid email, student ID / roll number, and password, ensuring unique email constraints.
FR-02	Password Security	The system shall hash all passwords using Bcrypt with 10 salt rounds prior to persisting them to the database.
FR-03	User Authentication	The system shall authenticate credentials and issue a signed, tamper-proof JWT bearer token expiring in 24 hours.
FR-04	Lost Item Reporting	The system shall allow authenticated students to file a lost item application with title, category, location, date, and description.
FR-05	Found Item Registration	The system shall permit users to report found articles, upload photos via Multer, and set an anti-fraud challenge question.
FR-06	Live Catalog Search	The system shall provide instantaneous full-text filtering across all active listings by keyword, campus building, and item type.
FR-07	Anti-Fraud Claim Filing	The system shall allow users to file a claim by answering the challenge question and attaching optional proof documents.
FR-08	Claim Evaluation	The item poster shall have exclusive access to view submitted claims and execute "Approve" or "Reject" status transitions.
FR-09	Notification System	The system shall generate automated in-app notifications to claimants when their claim is approved, rejected, or updated.

Req. ID	Feature Scope	Requirement Description & System Behavior
FR-10	Resolution Tracking	When a claim is approved, the item status shall transition to "CLAIMED" or "RESOLVED", removing it from active searches.
FR-11	Administrative Oversight	The system shall provide an admin console allowing security personnel to delete spam, view analytics, and inspect audit logs.
FR-12	Profile Management	The system shall allow users to view their filed listings, active claims, and manage account details in a personal dashboard.

4.4 Non-Functional Requirements

- Performance: Public catalog queries and full-text searches shall return results in less than 300 milliseconds under local server execution.
- Security: All REST API endpoints modifying state shall enforce JWT token validation and role-based permissions (Student vs Admin).
- Portability: The entire application, backend API, and database shall execute seamlessly on Windows, Linux, and macOS without requiring external cloud database subscriptions.
- Responsiveness: The UI layout shall dynamically adapt to screen resolutions ranging from 320px mobile smartphones up to 4K ultra-wide monitors.
- Maintainability: Clean decoupling of frontend client code, backend REST controllers, and Prisma schema definitions ensures effortless code maintainability.

CHAPTER-5

5. EXISTING SYSTEM

5.1 Existing System Overview

In prevailing educational environments, the handling of lost and found property is entirely informal and manual. Institutions depend on security guard handbooks, corkboards, and unstructured social media threads.

- Manual Security Registers: Handwritten logbooks at security gates where guards record brief notes.
- Campus Notice Boards: Paper flyers pinned on cork bulletin boards in departmental corridors.
- Student WhatsApp Groups: Unofficial, unmoderated group chats where students post lost notices.

5.2 Problems in the Existing System

- No Centralized Storage or Indexing: An item found in the Science Laboratory remains unknown to security personnel at the Main Entrance Gate.
- High Susceptibility to Opportunistic Theft: In open WhatsApp groups, opportunistic individuals falsely claim lost valuables because finders cannot verify true ownership.
- Privacy Compromise: Students post their personal mobile phone numbers on physical flyers, leading to unsolicited calls and privacy violations.
- Lack of Visual Documentation: Handwritten paper logbooks cannot store photographs, resulting in ambiguous entries like "black bag" that confuse students.

5.3 Limitations Summary & Comparative Matrix

Evaluation Metric	Manual Security Book	College Chat Groups	CampusTrack (Our System)
Searchability	None (Manual leafing)	Poor keyword scrolling	Instant Real-Time Search
Photo Storage	Impossible	Unindexed image spam	High-Res Linked Photos
Ownership Proof	None	None (First to claim wins)	Anti-Fraud Challenge Question
Student Privacy	Public ledger view	Mobile numbers exposed	Mediated Secure Contact
Campus Coverage	Single physical gate	Fragmented batch chats	Unified Campus-Wide Portal
Administrative Audit	Incomplete records	Zero institutional audit	Complete Relational Audit Log

CHAPTER-6

6. PROPOSED SYSTEM

6.1 Proposed System Overview

CampusTrack establishes an end-to-end digital recovery infrastructure. It unifies students, faculty, and campus security officers into a single, cohesive ecosystem.

6.2 Key Architectural Decisions

- Modern Single Page Application: Built using React 19 and Tailwind CSS v4, providing an app-like experience with immediate UI feedback and zero page reloads.
- Decoupled RESTful API: An Express 5 backend delivering clean JSON endpoints, allowing future native mobile apps to connect to the exact same backend.
- Embedded Relational Database: SQLite managed via Prisma ORM provides ACID-compliant transactions with zero external database configuration.
- Local Multipart Image Streaming: High-resolution photos are handled via Multer and served locally, eliminating expensive third-party storage fees.

6.3 Anti-Fraud Challenge Workflow

The distinguishing engineering feature of CampusTrack is its Anti-Fraud Security Challenge Protocol:

- Step 1: Finder Discovers Item (e.g., Branded Wallet, Keys, or Headphones).
- Step 2: Finder Submits Post + Challenge Question ("What photo is inside the clear ID slot?").
- Step 3: Public Views Item Listing (Image & Details Displayed; Challenge Active).
- Step 4: Legitimate Owner Submits Claim + Specific Hidden Answer + Bill Photo.
- Step 5: Poster Receives Confidential Notification & Evaluates Answer.
- Step 6: Poster Clicks [APPROVE] -> Safe Exchange Mediated at Campus Security Desk.

6.4 Feasibility Study

Feasibility Dimension	Assessment & Justification
Technical Feasibility	The stack (React 19, Node.js, Express, Prisma, SQLite) is mature, well-documented, highly stable, and runs on standard campus computer hardware without requiring specialized GPU or cloud clusters.
Operational Feasibility	Students and campus security personnel are already familiar with modern web interfaces. The UI requires zero formal training, featuring clear visual buttons, category filters, and intuitive prompts.
Economic Feasibility	The application utilizes 100% open-source software libraries. Zero licensing fees, cloud subscription costs, or proprietary database charges are incurred.
Schedule Feasibility	The project was systematically developed across four agile sprints over the academic semester, achieving complete implementation and verification on schedule.

CHAPTER-7

7. SYSTEM ANALYSIS & DESIGN

7.1 Tiered System Architecture

CampusTrack follows a 3-tier decoupled client-server-database architecture:

- Presentation Tier (Client): React 19 SPA, Tailwind CSS v4, Axios, Lucide Icons.
- Application Tier (Server): Node.js Runtime, Express.js 5 REST Framework, JWT Authentication & Password Hashing, Multer File Upload & MIME Inspection, Anti-Fraud Challenge & Claim State Machine.
- Data Tier (Persistence): Prisma ORM 6.4.1 Client, Embedded SQLite 3 (dev.db) Relational Store.

7.2 Data Design & Database Entities

Table Name	Primary Key	Foreign Keys	Core Attributes
User	id (UUID)	None	email (Unique), studentId, password, name, phone, role (STUDENT/ADMIN), createdAt
Item	id (UUID)	userId -> User(id)	title, description, category, type (LOST/FOUND), location, date, imageUrl, question, status (OPEN/CLAIMED/RESOLVED)
Claim	id (UUID)	itemId -> Item(id), claimantId -> User(id)	answer, proofUrl, message, status (PENDING/APPROVED/REJECTED), createdAt
Notification	id (UUID)	userId -> User(id)	title, message, type, isRead (Boolean), createdAt

7.4 Detailed Use Case Specifications

The core interactions between campus actors (Students, Finders, Claimants, and Security Personnel) and CampusTrack are specified below:

Use Case ID	Actor	Trigger Event	Main Success Scenario
UC-01	Student	Student accesses /register	Enters roll number, college email, name, and password. System hashes password via Bcrypt and stores record in SQLite.
UC-02	Student	Student clicks "Report Lost Item"	Fills lost item title, category, campus building (e.g., Library), date, and contact details. Item status initialized to OPEN.
UC-03	Finder	Finder clicks "Report Found Item"	Fills found item attributes, uploads high-res photo via Multer, and defines an Anti-Fraud Security Challenge Question.
UC-04	Claimant	Claimant clicks "Submit Claim"	Provides answer to concealed challenge question and uploads optional purchase bill/photo. Claim logged with PENDING status.
UC-05	Poster	Poster reviews claim in Dashboard	Inspects claimant answer and attached proof. Clicks [Approve Claim]. Claim becomes APPROVED and item transitions to CLAIMED.
UC-06	Security Admin	Admin accesses /admin	Views system statistics, audits student directory, inspects unresolved disputes, and permanently deletes fraudulent listings.

7.5 Relational Data Dictionary

The complete attribute dictionary across all SQLite tables managed by Prisma ORM:

Table	Attribute Name	Data Type	Constraints	Functional Description
User	id	String (UUID)	PRIMARY KEY	Unique identifier for student/admin.
User	email	String	UNIQUE, NOT NULL	University email address for authentication.
User	studentId	String	NOT NULL	College student roll number or faculty code.

Table	Attribute Name	Data Type	Constraints	Functional Description
User	password	String	NOT NULL	Bcrypt hashed password digest (10 rounds).
User	role	String	DEFAULT "STUDENT"	Access role: "STUDENT" or "ADMIN".
Item	id	String (UUID)	PRIMARY KEY	Unique identifier for item post.
Item	title	String	NOT NULL	Brief headline describing the misplaced item.
Item	type	String	NOT NULL	"LOST" or "FOUND" application classification.
Item	category	String	NOT NULL	Category: Electronics, IDs, Keys, Books, Bags.
Item	location	String	NOT NULL	Specific campus facility (e.g., Computer Lab 3).
Item	imageUrl	String	OPTIONAL	Relative file path in /uploads storage.
Item	question	String	OPTIONAL	Concealed anti-fraud ownership challenge.
Item	status	String	DEFAULT "OPEN"	"OPEN", "CLAIM_PENDING", "CLAIMED", "RESOLVED".
Item	userId	String (UUID)	FOREIGN KEY	References User(id) on cascade delete.
Claim	id	String (UUID)	PRIMARY KEY	Unique identifier for ownership claim.
Claim	itemId	String (UUID)	FOREIGN KEY	References Item(id) on cascade delete.
Claim	claimantId	String (UUID)	FOREIGN KEY	References User(id) on cascade delete.
Claim	answer	String	NOT NULL	Claimant answer to challenge question.
Claim	proofUrl	String	OPTIONAL	Attached receipt or ownership photograph.
Claim	status	String	DEFAULT "PENDING"	"PENDING", "APPROVED", "REJECTED".
Notification	id	String (UUID)	PRIMARY KEY	Unique notification alert identifier.
Notification	userId	String (UUID)	FOREIGN KEY	References recipient User(id).
Notification	isRead	Boolean	DEFAULT false	Read/unread delivery status indicator.

7.6 Item State Transition Lifecycle

Lost and found records progress through a strict state machine to prevent race conditions:

- OPEN: Newly submitted item available in public search and browsing catalog.
- CLAIM_PENDING: One or more claimants have submitted challenge answers under review.
- CLAIMED: Poster has approved a legitimate claim; physical handover scheduled at security desk.
- RESOLVED: Item safely reunited with owner; permanently archived in university audit logs.

7.3 RESTful API Design Specification

HTTP Method	Endpoint Route	Auth Required	Functional Description
POST	/api/auth/register	No	Registers new student account with hashed password.
POST	/api/auth/login	No	Authenticates credentials and returns JWT token.
GET	/api/items	No	Fetches all active lost/found items with search query filters.
POST	/api/items	Yes (Bearer JWT)	Uploads photo via Multer and creates item listing.
GET	/api/items/:id	No	Retrieves detailed item record and challenge question.
POST	/api/items/:id/claim	Yes (Bearer JWT)	Submits ownership challenge answer and proof document.

HTTP Method	Endpoint Route	Auth Required	Functional Description
PATCH	/api/claims/:id	Yes (Poster/Admin)	Updates claim status to APPROVED or REJECTED.
GET	/api/admin/stats	Yes (Admin Role)	Returns system statistics and recovery metric counters.
DELETE	/api/admin/items/:id	Yes (Admin Role)	Deletes spam listing and associated photo from disk.

CHAPTER-8

8. METHODOLOGY

8.1 SDLC Methodology (Agile & Iterative Lifecycle)

The development of CampusTrack followed the Agile Iterative Development Lifecycle across four structured sprints:

Sprint / Phase	Primary Focus	Milestones & Deliverables
Sprint 1	Scaffolding & Auth	Prisma schema modeling, SQLite database migration, Express 5 server setup, Bcrypt password hashing, and JWT bearer authentication.
Sprint 2	Item Ingestion & Uploads	Multer multipart file streaming engine, React dual reporting forms, category taxonomy definition, and local image serving.
Sprint 3	Verification & Claims	Anti-Fraud Challenge question engine, claim modal, notification dispatcher, and poster approve/reject decision controls.
Sprint 4	Security Admin & Hardening	Campus security console, analytical metric counters, role-based authorization, responsive UI polish, and cross-platform runners.

8.2 Anti-Fraud Processing Logic

The anti-fraud engine ensures that claims are strictly evaluated through evidence rather than emotional or arbitrary claims:

- Attribute Masking: Critical identifying attributes (e.g., serial numbers, internal engravings, wallpaper images) are withheld from public view.
- Challenge Validation: The claimant must provide an exact or closely corroborating description of the hidden feature.
- Document Verification: Claimants can attach purchase receipts, serial number cards, or earlier personal photos showing the item in their possession.
- Audit Trail: All approved and rejected claims are permanently preserved in the relational database for administrative inspection.

CHAPTER-9

9. SYSTEM IMPLEMENTATION & SOURCE CODE LISTINGS

This chapter documents the technical realization of CampusTrack, presenting the actual production source code across the database schema, backend REST API, authentication middleware, file ingestion pipeline, and frontend React components.

9.1 Database Modeling with Prisma ORM

The complete relational data model defined in schema.prisma enforces cascading deletes and referential integrity between users, items, and verification claims:

Listing: server/prisma/schema.prisma

```
datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

model User {
  id           Int            @id @default(autoincrement())
  name         String
  email        String         @unique
  password     String
  phone        String?
  collegeId    String?
  role         String         @default("STUDENT") // STUDENT or ADMIN
  createdAt    DateTime       @default(now())
  updatedAt    DateTime       @updatedAt

  items        Item[]
  claims       Claim[]
  notifications Notification[]
}

model Item {
  id               Int       @id @default(autoincrement())
  title            String
  description      String
  category         String    // "Electronics", "ID Cards", "Keys", "Wallets & Bags",
"Books & Stationery", "Clothing", "Other"
  type             String    // "LOST" or "FOUND"
  status           String    @default("OPEN") // "OPEN", "CLAIM_PENDING", "RESOLVED"
  location         String    // e.g. "Library 2nd Floor", "Canteen", "Room 304", etc.
  date             DateTime
  imageUrl         String?
  securityQuestion String?   // For FOUND items (anti-fraud claim verification)
  contactEmail     String?
  contactPhone     String?

  userId           Int
  user             User      @relation(fields: [userId], references: [id], onDelete:
Cascade)

  claims           Claim[]
  createdAt        DateTime  @default(now())
  updatedAt        DateTime  @updatedAt
}

model Claim {
  id              Int       @id @default(autoincrement())
  itemId          Int
  item            Item      @relation(fields: [itemId], references: [id], onDelete:
Cascade)
  claimantId      Int
  claimant        User      @relation(fields: [claimantId], references: [id], onDelete:
Cascade)
  status          String    @default("PENDING") // "PENDING", "APPROVED", "REJECTED"
  message         String    // Reason, description of hidden markings, or answer to
security question
```

```
  proofImageUrl   String?
  claimantContact String?
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt
}

model Notification {
  id        Int      @id @default(autoincrement())
  userId    Int
  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  title     String
  message   String
  link      String?
  isRead    Boolean  @default(false)
  createdAt DateTime @default(now())
}

```

9.2 Backend Server Initialization & HTTP Middleware

The server entry point (index.js) mounts CORS, JSON body parsers, static media directories, and routes:

Listing: server/src/index.js

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const path = require('path');

const authRoutes = require('./routes/authRoutes');
const itemRoutes = require('./routes/itemRoutes');
const claimRoutes = require('./routes/claimRoutes');
const adminRoutes = require('./routes/adminRoutes');

const app = express();
const PORT = process.env.PORT || 5000;

// Middlewares
app.use(cors({
  origin: true,
  credentials: true
}));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Serve uploaded photos statically
app.use('/uploads', express.static(path.join(__dirname, '../uploads')));

// Health check
app.get('/api/health', (req, res) => {
  res.json({ status: 'OK', message: 'College Lost & Found API is up and running!', timestamp: new Date() });
});

// API Routes
app.use('/api/auth', authRoutes);
app.use('/api/items', itemRoutes);
app.use('/api/claims', claimRoutes);
app.use('/api/admin', adminRoutes);

// Global Error Handler
app.use((err, req, res, next) => {
  console.error('Unhandled Server Error:', err);
  if (err instanceof multer?.MulterError) {
    return res.status(400).json({ message: `Upload error: ${err.message}` });
  }
  res.status(err.status || 500).json({
    message: err.message || 'Internal Server Error'
  });
});

app.listen(PORT, () => {
  console.log(`================================================`);
  console.log(`🎓 College Lost & Found Server running on port ${PORT}`);
  console.log(`🔗 API Base: http://localhost:${PORT}/api`);
  console.log(`📁 Static Uploads: http://localhost:${PORT}/uploads`);
  console.log(`================================================`);
});

```

9.3 Authentication & Authorization Middleware

The auth middleware verifies JSON Web Tokens (JWT) extracted from HTTP Authorization bearer headers:

Listing: server/src/middleware/auth.js

```
const jwt = require('jsonwebtoken');
const prisma = require('../prisma');

const authMiddleware = async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return res.status(401).json({ message: 'Authentication required. No token provided.' });
    }

    const token = authHeader.split(' ')[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET || 'secret');

    const user = await prisma.user.findUnique({
      where: { id: decoded.userId },
      select: { id: true, name: true, email: true, phone: true, collegeId: true, role: true }
    });

    if (!user) {
      return res.status(401).json({ message: 'User not found or deleted.' });
    }

    req.user = user;
    next();
  } catch (err) {
    return res.status(401).json({ message: 'Invalid or expired token.' });
  }
};

const optionalAuthMiddleware = async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;
    if (authHeader && authHeader.startsWith('Bearer ')) {
      const token = authHeader.split(' ')[1];
      const decoded = jwt.verify(token, process.env.JWT_SECRET || 'secret');
      const user = await prisma.user.findUnique({
        where: { id: decoded.userId },
        select: { id: true, name: true, email: true, phone: true, collegeId: true, role: true }
      });
      if (user) req.user = user;
    }
  } catch (err) {
    // Continue without user
  }
  next();
};

const adminOnly = (req, res, next) => {
  if (!req.user || req.user.role !== 'ADMIN') {
    return res.status(403).json({ message: 'Access denied: College administrator privilege required.' });
  }
  next();
};

module.exports = { authMiddleware, optionalAuthMiddleware, adminOnly };

```

9.4 Multipart Media Streaming with Multer

The upload middleware intercepts binary multipart/form-data image payloads, assigns unique timestamps, and validates MIME types:

Listing: server/src/middleware/upload.js

```
const multer = require('multer');
const path = require('path');
const fs = require('fs');

// Ensure upload directory exists
const uploadDir = path.join(__dirname, '../../uploads');
if (!fs.existsSync(uploadDir)) {
  fs.mkdirSync(uploadDir, { recursive: true });
}

const storage = multer.diskStorage({
  destination: function (req, file, cb) {
    cb(null, uploadDir);
  },
  filename: function (req, file, cb) {
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1e9);
    const ext = path.extname(file.originalname).toLowerCase();
    cb(null, 'item-' + uniqueSuffix + ext);
  }
});

const fileFilter = (req, file, cb) => {
  const allowedTypes = /jpeg|jpg|png|webp|gif/;
  const extname = allowedTypes.test(path.extname(file.originalname).toLowerCase());
  const mimetype = allowedTypes.test(file.mimetype);

  if (extname && mimetype) {
    return cb(null, true);
  } else {
    cb(new Error('Only image files (jpg, jpeg, png, webp, gif) are allowed!'));
  }
};

const upload = multer({
  storage: storage,
  limits: { fileSize: 5 * 1024 * 1024 }, // 5MB limit
  fileFilter: fileFilter
});

module.exports = upload;

```

9.5 User Authentication Controller

The authController handles student account registration with Bcrypt password hashing (10 salt rounds) and session login:

Listing: server/src/controllers/authController.js

```
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const prisma = require('../prisma');

const register = async (req, res) => {
  try {
    const { name, email, password, phone, collegeId } = req.body;

    if (!name || !email || !password) {
      return res.status(400).json({ message: 'Name, email, and password are required.' });
    }

    const existingUser = await prisma.user.findUnique({
      where: { email: email.toLowerCase().trim() }
    });

    if (existingUser) {
      return res.status(400).json({ message: 'An account with this email already exists.'
});
    }

    const hashedPassword = await bcrypt.hash(password, 10);

    // Auto-grant ADMIN role if email starts with admin@
    const role = email.toLowerCase().startsWith('admin@') ? 'ADMIN' : 'STUDENT';

    const user = await prisma.user.create({
      data: {
        name,
        email: email.toLowerCase().trim(),
        password: hashedPassword,
        phone: phone || null,
        collegeId: collegeId || null,
        role
      },
      select: {
        id: true,
        name: true,
        email: true,
        phone: true,
        collegeId: true,
        role: true,
        createdAt: true
      }
    });

    const token = jwt.sign(
      { userId: user.id, role: user.role },
      process.env.JWT_SECRET || 'secret',
      { expiresIn: '7d' }
    );

    res.status(201).json({
      message: 'Account created successfully',
      user,
      token
    });
  } catch (error) {
    console.error('Registration error:', error);
    res.status(500).json({ message: 'Server error during registration.' });
  }
};

const login = async (req, res) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({ message: 'Email and password are required.' });
    }

```

```
    const user = await prisma.user.findUnique({
      where: { email: email.toLowerCase().trim() }
    });

    if (!user) {
      return res.status(401).json({ message: 'Invalid email or password.' });
    }

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      return res.status(401).json({ message: 'Invalid email or password.' });
    }

    const token = jwt.sign(
      { userId: user.id, role: user.role },
      process.env.JWT_SECRET || 'secret',
      { expiresIn: '7d' }
    );

    res.json({
      message: 'Logged in successfully',
      user: {
        id: user.id,
        name: user.name,
        email: user.email,
        phone: user.phone,
        collegeId: user.collegeId,
        role: user.role,
        createdAt: user.createdAt
      },
      token
    });
  } catch (error) {
    console.error('Login error:', error);
    res.status(500).json({ message: 'Server error during login.' });
  }
};

const getMe = async (req, res) => {
  try {
    const user = await prisma.user.findUnique({
      where: { id: req.user.id },
      select: {
        id: true,
        name: true,
        email: true,
        phone: true,
        collegeId: true,
        role: true,
        createdAt: true,
        _count: {
          select: {
            items: true,
            claims: true
          }
        }
      }
    });

    res.json({ user });
  } catch (error) {
    res.status(500).json({ message: 'Error fetching profile.' });
  }
};

const updateProfile = async (req, res) => {
  try {
    const { name, phone, collegeId } = req.body;
    const updated = await prisma.user.update({
      where: { id: req.user.id },
```

9.6 Lost & Found Item Management Controller

The itemController powers dual application filing, category querying, full-text search, and detail retrieval:

Listing: server/src/controllers/itemController.js

```
const prisma = require('../prisma');
const fs = require('fs');
const path = require('path');

// Get all items with rich filtering & search
const getItems = async (req, res) => {
  try {
    const { type, category, status, search, location, sort = 'newest' } = req.query;

    const where = {};

    if (type && type !== 'ALL') {
      where.type = type.toUpperCase();
    }

    if (category && category !== 'ALL') {
      where.category = category;
    }

    if (status && status !== 'ALL') {
      where.status = status.toUpperCase();
    }

    if (location) {
      where.location = { contains: location };
    }

    if (search) {
      where.OR = [
        { title: { contains: search } },
        { description: { contains: search } },
        { location: { contains: search } },
        { category: { contains: search } }
      ];
    }

    let orderBy = { createdAt: 'desc' };
    if (sort === 'oldest') orderBy = { createdAt: 'asc' };
    if (sort === 'date_desc') orderBy = { date: 'desc' };
    if (sort === 'date_asc') orderBy = { date: 'asc' };

    const items = await prisma.item.findMany({
      where,
      orderBy,
      include: {
        user: {
          select: { id: true, name: true, email: true, phone: true, collegeId: true }
        },
        _count: {
          select: { claims: true }
        }
      }
    });

    res.json({ items, total: items.length });
  } catch (error) {
    console.error('Error fetching items:', error);
    res.status(500).json({ message: 'Error retrieving items.' });
  }
};

// Get single item by ID
const getItemById = async (req, res) => {
  try {
    const { id } = req.params;
    const itemId = parseInt(id, 10);

    const item = await prisma.item.findUnique({
      where: { id: itemId },
      include: {
        user: {
```

```
            select: { id: true, name: true, email: true, phone: true, collegeId: true }
          },
          claims: {
            include: {
              claimant: {
                select: { id: true, name: true, email: true, phone: true, collegeId: true }
              }
            },
            orderBy: { createdAt: 'desc' }
          }
        }
      });

      if (!item) {
        return res.status(404).json({ message: 'Item not found.' });
      }

      // Mask claims info if current viewer is neither the item owner nor an admin
      const currentUserId = req.user ? req.user.id : null;
      const isOwnerOrAdmin = req.user && (req.user.id === item.userId || req.user.role ===
'ADMIN');

      if (!isOwnerOrAdmin) {
        // Filter out claims that don't belong to the logged-in viewer
        item.claims = item.claims.filter(c => c.claimantId === currentUserId);
      }

      res.json({ item });
    } catch (error) {
      console.error('Error fetching item details:', error);
      res.status(500).json({ message: 'Error retrieving item details.' });
    }
  };

  // Create new Lost or Found item
  const createItem = async (req, res) => {
    try {
      const {
        title,
        description,
        category,
        type,
        location,
        date,
        securityQuestion,
        contactEmail,
        contactPhone
      } = req.body;

      if (!title || !description || !category || !type || !location || !date) {
        return res.status(400).json({ message: 'Please provide all required fields.' });
      }

      let imageUrl = null;
      if (req.file) {
        imageUrl = `/uploads/${req.file.filename}`;
      }

      const item = await prisma.item.create({
        data: {
          title,
          description,
          category,
          type: type.toUpperCase(),
          location,
          date: new Date(date),
          imageUrl,
          securityQuestion: securityQuestion || null,
          contactEmail: contactEmail || req.user.email,
          contactPhone: contactPhone || req.user.phone,
```

9.7 Anti-Fraud Claim & Verification Controller

The claimController enforces the challenge-question verification algorithm, evaluates proof documents, and executes claim status transitions:

Listing: server/src/controllers/claimController.js

```
const prisma = require('../prisma');

// Submit a claim on an item
const createClaim = async (req, res) => {
  try {
    const { itemId, message, claimantContact } = req.body;

    if (!itemId || !message) {
      return res.status(400).json({ message: 'Item ID and verification message/description
are required.' });
    }

    const item = await prisma.item.findUnique({
      where: { id: parseInt(itemId, 10) },
      include: { user: true }
    });

    if (!item) {
      return res.status(404).json({ message: 'Item not found.' });
    }

    if (item.userId === req.user.id) {
      return res.status(400).json({ message: 'You cannot submit a claim on your own posted
item.' });
    }

    // Check if user already claimed this item
    const existingClaim = await prisma.claim.findFirst({
      where: {
        itemId: item.id,
        claimantId: req.user.id
      }
    });

    if (existingClaim) {
      return res.status(400).json({ message: 'You have already submitted a claim on this
item.' });
    }

    let proofImageUrl = null;
    if (req.file) {
      proofImageUrl = `/uploads/${req.file.filename}`;
    }

    const claim = await prisma.claim.create({
      data: {
        itemId: item.id,
        claimantId: req.user.id,
        message,
        claimantContact: claimantContact || req.user.phone || req.user.email,
        proofImageUrl
      },
      include: {
        claimant: {
          select: { id: true, name: true, email: true, phone: true }
        }
      }
    });

    // Notify item owner
    await prisma.notification.create({
      data: {
        userId: item.userId,
        title: 'New Claim Received',
        message: `${req.user.name} submitted a claim request for "${item.title}". Check
your dashboard to verify.`,
        link: `/items/${item.id}`
      }
    });

    // Update item status to CLAIM_PENDING if currently OPEN
```

```
      if (item.status === 'OPEN') {
        await prisma.item.update({
          where: { id: item.id },
          data: { status: 'CLAIM_PENDING' }
        });
      }

      res.status(201).json({
        message: 'Claim request submitted successfully! The poster has been notified.',
        claim
      });
    } catch (error) {
      console.error('Error submitting claim:', error);
      res.status(500).json({ message: 'Server error while submitting claim.' });
    }
  };

  // Update claim status (Approve / Reject) by item owner or Admin
  const updateClaimStatus = async (req, res) => {
    try {
      const { id } = req.params;
      const { status } = req.body; // "APPROVED" or "REJECTED"

      if (!['APPROVED', 'REJECTED'].includes(status)) {
        return res.status(400).json({ message: 'Invalid status. Must be APPROVED or
REJECTED.' });
      }

      const claim = await prisma.claim.findUnique({
        where: { id: parseInt(id, 10) },
        include: {
          item: true,
          claimant: true
        }
      });

      if (!claim) {
        return res.status(404).json({ message: 'Claim not found.' });
      }

      const isOwnerOrAdmin = claim.item.userId === req.user.id || req.user.role === 'ADMIN';
      if (!isOwnerOrAdmin) {
        return res.status(403).json({ message: 'Unauthorized. Only item poster or admin can
review claims.' });
      }

      const updatedClaim = await prisma.claim.update({
        where: { id: claim.id },
        data: { status }
      });

      if (status === 'APPROVED') {
        // Mark item as RESOLVED
        await prisma.item.update({
          where: { id: claim.itemId },
          data: { status: 'RESOLVED' }
        });

        // Reject all other pending claims on this item
        await prisma.claim.updateMany({
          where: {
            itemId: claim.itemId,
            id: { not: claim.id },
            status: 'PENDING'
          },
          data: { status: 'REJECTED' }
        });

        // Notify claimant with contact instructions
        await prisma.notification.create({
          data: {
            userId: claim.claimantId,
            title: 'Claim Approved! 🎉',
            message: `Great news! Your claim for "${claim.item.title}" was approved. You can
contact the owner to collect it.`,
            link: `/items/${claim.itemId}`
```

9.8 Campus Security Administration Controller

The adminController provides campus security officers with operational statistics and moderation endpoints:

Listing: server/src/controllers/adminController.js

```
const prisma = require('../prisma');

// Admin campus stats
const getAdminOverview = async (req, res) => {
  try {
    const [
      totalUsers,
      totalItems,
      totalLost,
      totalFound,
      totalResolved,
      totalClaims,
      pendingClaims
    ] = await Promise.all([
      prisma.user.count(),
      prisma.item.count(),
      prisma.item.count({ where: { type: 'LOST' } }),
      prisma.item.count({ where: { type: 'FOUND' } }),
      prisma.item.count({ where: { status: 'RESOLVED' } }),
      prisma.claim.count(),
      prisma.claim.count({ where: { status: 'PENDING' } })
    ]);

    const users = await prisma.user.findMany({
      select: {
        id: true,
        name: true,
        email: true,
        phone: true,
        collegeId: true,
        role: true,
        createdAt: true,
        _count: { select: { items: true, claims: true } }
      },
      orderBy: { createdAt: 'desc' }
    });

    const recentClaims = await prisma.claim.findMany({
      take: 10,
      orderBy: { createdAt: 'desc' },
      include: {
        item: { select: { id: true, title: true, type: true } },
        claimant: { select: { id: true, name: true, email: true } }
      }
    });

    res.json({
      stats: {
        totalUsers,
        totalItems,
        totalLost,
        totalFound,
        totalResolved,
        totalClaims,
        pendingClaims
      },
      users,
      recentClaims
    });
  } catch (error) {
    console.error('Admin overview error:', error);
    res.status(500).json({ message: 'Error retrieving admin metrics.' });
  }
};

// Change user role
const updateUserRole = async (req, res) => {
  try {
    const { id } = req.params;
    const { role } = req.body;

```

```
    if (!['STUDENT', 'ADMIN'].includes(role)) {
      return res.status(400).json({ message: 'Invalid role. Must be STUDENT or ADMIN.' });
    }

    const updated = await prisma.user.update({
      where: { id: parseInt(id, 10) },
      data: { role },
      select: { id: true, name: true, email: true, role: true }
    });

    res.json({ message: `User role updated to ${role}.`, user: updated });
  } catch (error) {
    res.status(500).json({ message: 'Error updating user role.' });
  }
};

// Admin delete any item
const adminDeleteItem = async (req, res) => {
  try {
    const { id } = req.params;
    await prisma.item.delete({ where: { id: parseInt(id, 10) } });
    res.json({ message: 'Item deleted by college administrator.' });
  } catch (error) {
    res.status(500).json({ message: 'Error deleting item.' });
  }
};

module.exports = {
  getAdminOverview,
  updateUserRole,
  adminDeleteItem
};

```

9.9 RESTful API Routing Modules

The Express routing modules map HTTP verbs to controller actions:

Listing: server/src/routes/itemRoutes.js

```
const express = require('express');
const router = express.Router();
const {
  getItems,
  getItemById,
  createItem,
  updateItem,
  deleteItem,
  getMyItems,
  getStats
} = require('../controllers/itemController');
const { authMiddleware, optionalAuthMiddleware } = require('../middleware/auth');
const upload = require('../middleware/upload');

// Public routes (or with optional auth to customize claim visibility)
router.get('/stats', getStats);
router.get('/', getItems);
router.get('/my', authMiddleware, getMyItems);
router.get('/:id', optionalAuthMiddleware, getItemById);

// Protected routes
router.post('/', authMiddleware, upload.single('image'), createItem);
router.put('/:id', authMiddleware, upload.single('image'), updateItem);
router.delete('/:id', authMiddleware, deleteItem);

module.exports = router;

```

Listing: server/src/routes/claimRoutes.js

```
const express = require('express');
const router = express.Router();
const {
  createClaim,
  updateClaimStatus,
  getMyClaims,
  getNotifications,
  markNotificationRead
} = require('../controllers/claimController');
const { authMiddleware } = require('../middleware/auth');
const upload = require('../middleware/upload');

router.post('/', authMiddleware, upload.single('proofImage'), createClaim);
router.put('/:id/status', authMiddleware, updateClaimStatus);
router.get('/my', authMiddleware, getMyClaims);
router.get('/notifications', authMiddleware, getNotifications);
router.put('/notifications/:id/read', authMiddleware, markNotificationRead);

module.exports = router;

```

Listing: server/src/routes/authRoutes.js

```
const express = require('express');
const router = express.Router();
const { register, login, getMe, updateProfile } =
require('../controllers/authController');
const { authMiddleware } = require('../middleware/auth');

router.post('/register', register);
router.post('/login', login);
router.get('/me', authMiddleware, getMe);
router.put('/profile', authMiddleware, updateProfile);

module.exports = router;

```

Listing: server/src/routes/adminRoutes.js

```
const express = require('express');
const router = express.Router();
const {
  getAdminOverview,
  updateUserRole,
  adminDeleteItem
```

```
} = require('../controllers/adminController');
const { authMiddleware, adminOnly } = require('../middleware/auth');

router.use(authMiddleware, adminOnly);

router.get('/overview', getAdminOverview);
router.put('/users/:id/role', updateUserRole);
router.delete('/items/:id', adminDeleteItem);

module.exports = router;

```

9.10 Automated Database Seeding Engine

The seed.js script populates initial demonstration listings, test student accounts, and admin credentials for academic examination:

Listing: server/src/seed.js

```
const bcrypt = require('bcryptjs');
const prisma = require('./prisma');

async function seed() {
  console.log('🍸 Starting database seed for College Lost & Found...');

  // Clean existing data
  await prisma.notification.deleteMany();
  await prisma.claim.deleteMany();
  await prisma.item.deleteMany();
  await prisma.user.deleteMany();

  const hashedPassword = await bcrypt.hash('password123', 10);
  const adminPassword = await bcrypt.hash('admin123', 10);

  // 1. Create Users
  const admin = await prisma.user.create({
    data: {
      name: 'Campus Security Desk',
      email: 'admin@college.edu',
      password: adminPassword,
      phone: '+91 98765 00001',
      collegeId: 'SEC-DESK-01',
      role: 'ADMIN'
    }
  });

  const rahul = await prisma.user.create({
    data: {
      name: 'Rahul Sharma',
      email: 'rahul.s@college.edu',
      password: hashedPassword,
      phone: '+91 98765 11111',
      collegeId: 'CS2023-042',
      role: 'STUDENT'
    }
  });

  const priya = await prisma.user.create({
    data: {
      name: 'Priya Patel',
      email: 'priya.p@college.edu',
      password: hashedPassword,
      phone: '+91 98765 22222',
      collegeId: 'EC2022-019',
      role: 'STUDENT'
    }
  });

  const ananya = await prisma.user.create({
    data: {
      name: 'Ananya Verma',
      email: 'ananya.v@college.edu',
      password: hashedPassword,
      phone: '+91 98765 33333',
      collegeId: 'ME2024-088',
      role: 'STUDENT'
    }
  });

  console.log('☑ Users seeded');

  // 2. Create Items
  const item1 = await prisma.item.create({
    data: {
      title: 'Casio fx-991EX Scientific Calculator',
      description: 'Black scientific calculator with a small scratch on the solar panel
corner. Left on table 4 during Mathematics mid-term examination.',
      category: 'Electronics',
      type: 'LOST',
```

```
        status: 'OPEN',
        location: 'Academic Block B - Room 302',
        date: new Date('2026-10-25'),
        contactEmail: rahul.email,
        contactPhone: rahul.phone,
        userId: rahul.id
      }
    });

    const item2 = await prisma.item.create({
      data: {
        title: 'College ID Card & Metro Pass',
        description: 'Lanyard with University ID card (Name: Priya Patel, Dept of ECE) along with a Blue Metro smart card inside the sleeve.',
        category: 'ID Cards',
        type: 'LOST',
        status: 'OPEN',
        location: 'Central Library - 2nd Floor Reading Hall',
        date: new Date('2026-10-26'),
        contactEmail: priya.email,
        contactPhone: priya.phone,
        userId: priya.id
      }
    });

    const item3 = await prisma.item.create({
      data: {
        title: 'Silver Bike Key with Spider-Man Keychain',
        description: 'Found a single motorbike key (Hero Splendor) attached to a metallic Spider-Man emblem keychain near the two-wheeler parking lot.',
        category: 'Keys',
        type: 'FOUND',
        status: 'OPEN',
        location: 'Student Parking Area (Zone 2)',
        date: new Date('2026-10-27'),
        securityQuestion: 'What text or number is engraved on the back of the key, or what color is the ring?',
        contactEmail: ananya.email,
        contactPhone: ananya.phone,
        userId: ananya.id
      }
    });

    const item4 = await prisma.item.create({
      data: {
        title: 'Navy Blue Fastrack Water Bottle (750ml)',
        description: 'Insulated steel bottle in dark navy blue. Found on the front bench after Mechanical Engineering lecture.',
        category: 'Other',
        type: 'FOUND',
        status: 'RESOLVED',
        location: 'Lecture Hall Complex - LH 104',
        date: new Date('2026-10-24'),
        securityQuestion: 'Are there any stickers on the base of the bottle?',
        contactEmail: admin.email,
        contactPhone: admin.phone,
        userId: admin.id
      }
    });

    const item5 = await prisma.item.create({
      data: {
        title: 'Boat Rockerz 450 Bluetooth Headphones',
        description: 'Matte black over-ear Bluetooth headphones left in the sports arena spectator stands during evening basketball match.',
        category: 'Electronics',
        type: 'LOST',
        status: 'CLAIM_PENDING',
        location: 'Sports Complex - Indoor Court',
        date: new Date('2026-10-28'),
        contactEmail: rahul.email,
        contactPhone: rahul.phone,
        userId: rahul.id
      }
    });

```

9.11 Frontend Application Router (React 19)

The client entry component App.jsx configures declarative routes using React Router and Tailwind styling:

Listing: client/src/App.jsx

```jsx
import React from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider, useAuth } from './context/AuthContext';
import Navbar from './components/Navbar';
import Footer from './components/Footer';

import Home from './pages/Home';
import BrowseItems from './pages/BrowseItems';
import ItemDetails from './pages/ItemDetails';
import CreateItem from './pages/CreateItem';
import Dashboard from './pages/Dashboard';
import AdminDashboard from './pages/AdminDashboard';
import Login from './pages/Login';
import Register from './pages/Register';

// Protected Route Wrapper
const ProtectedRoute = ({ children }) => {
  const { user, loading } = useAuth();
  if (loading) {
    return <div className="p-12 text-center text-xs text-slate-400">Loading...</div>;
  }
  if (!user) {
    return <Navigate to="/login" replace />;
  }
  return children;
};

// Admin Route Wrapper
const AdminRoute = ({ children }) => {
  const { user, loading } = useAuth();
  if (loading) {
    return <div className="p-12 text-center text-xs text-slate-400">Loading...</div>;
  }
  if (!user || user.role !== 'ADMIN') {
    return <Navigate to="/" replace />;
  }
  return children;
};

export default function App() {
  return (
    <AuthProvider>
      <Router>
        <div className="min-h-screen flex flex-col bg-slate-50/60 font-
['Plus_Jakarta_Sans',sans-serif]">
          <Navbar />

          <main className="flex-1">
            <Routes>
              <Route path="/" element={<Home />} />
              <Route path="/items" element={<BrowseItems />} />
              <Route path="/items/:id" element={<ItemDetails />} />
              <Route
                path="/create"
                element={
                  <ProtectedRoute>
                    <CreateItem />
                  </ProtectedRoute>
                }
              />
              <Route
                path="/dashboard"
                element={
                  <ProtectedRoute>
                    <Dashboard />
                  </ProtectedRoute>
                }
              />
              <Route
                path="/admin"
                element={
```

```
                    <AdminRoute>
                      <AdminDashboard />
                    </AdminRoute>
                  }
                />
                <Route path="/login" element={<Login />} />
                <Route path="/register" element={<Register />} />
                <Route path="*" element={<Navigate to="/" replace />} />
              </Routes>
            </main>

            <Footer />
          </div>
        </Router>
      </AuthProvider>
    );
  }

```

9.12 Global Authentication Context State

The AuthContext provides student authentication state, JWT storage in localStorage, and session persistence:

Listing: client/src/context/AuthContext.jsx

```
import React, { createContext, useContext, useState, useEffect } from 'react';
import api from '../api/client';

const AuthContext = createContext(null);

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(localStorage.getItem('college_token') || null);
  const [loading, setLoading] = useState(true);

  // Fetch current user on mount or token change
  useEffect(() => {
    const fetchMe = async () => {
      if (!token) {
        setUser(null);
        setLoading(false);
        return;
      }
      try {
        const res = await api.get('/auth/me');
        setUser(res.data.user);
      } catch (err) {
        console.error('Failed to load user profile:', err);
        localStorage.removeItem('college_token');
        setToken(null);
        setUser(null);
      } finally {
        setLoading(false);
      }
    };

    fetchMe();
  }, [token]);

  const login = async (email, password) => {
    const res = await api.post('/auth/login', { email, password });
    const { token: newToken, user: userData } = res.data;
    localStorage.setItem('college_token', newToken);
    setToken(newToken);
    setUser(userData);
    return userData;
  };

  const register = async (userData) => {
    const res = await api.post('/auth/register', userData);
    const { token: newToken, user: newUser } = res.data;
    localStorage.setItem('college_token', newToken);
    setToken(newToken);
    setUser(newUser);
    return newUser;
  };

  const logout = () => {
    localStorage.removeItem('college_token');
    setToken(null);
    setUser(null);
  };

  const refreshUser = async () => {
    if (!token) return;
    try {
      const res = await api.get('/auth/me');
      setUser(res.data.user);
    } catch (err) {
      console.error(err);
    }
  };

  return (
    <AuthContext.Provider value={{ user, token, loading, login, register, logout,
refreshUser }}>
```

```
71          {children}
72        </AuthContext.Provider>
73      );
74    };
75
76    export const useAuth = () => {
77      const context = useContext(AuthContext);
78      if (!context) {
79        throw new Error('useAuth must be used within an AuthProvider');
80      }
81      return context;
82    };
83
```

9.13 Homepage View Implementation

The Home.jsx page presents live recovery counters, action callout buttons, and the campus search hub:

Listing: client/src/pages/Home.jsx

```
import React, { useState, useEffect } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import api from '../api/client';
import ItemCard from '../components/ItemCard';
import {
  Search,
  PlusCircle,
  ShieldCheck,
  MapPin,
  Clock,
  ArrowRight,
  HelpCircle,
  CheckCircle2,
  Sparkles,
  AlertCircle
} from 'lucide-react';

export default function Home() {
  const navigate = useNavigate();
  const [stats, setStats] = useState({
    totalLost: 0,
    totalFound: 0,
    totalResolved: 0,
    totalUsers: 0,
    recentItems: []
  });
  const [searchQuery, setSearchQuery] = useState('');
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    loadHomeData();
  }, []);

  const loadHomeData = async () => {
    try {
      const res = await api.get('/items/stats');
      setStats(res.data);
    } catch (err) {
      console.error('Failed to load stats:', err);
    } finally {
      setLoading(false);
    }
  };

  const handleSearchSubmit = (e) => {
    e.preventDefault();
    if (searchQuery.trim()) {
      navigate(`/items?search=${encodeURIComponent(searchQuery.trim())}`);
    } else {
      navigate('/items');
    }
  };

  return (
    <div className="space-y-16">

      {/* Hero Section */}
      <section className="relative overflow-hidden pt-12 pb-16 md:pt-20 md:pb-24">
        {/* Subtle Background Glow */}
        <div className="absolute top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2 w-
[600px] h-[300px] bg-gradient-to-tr from-indigo-200/40 via-violet-200/30 to-emerald-200/20
blur-3xl rounded-full -z-10 pointer-events-none"></div>

        <div className="max-w-4xl mx-auto text-center px-4">
          <div className="inline-flex items-center gap-2 px-3 py-1 rounded-full bg-indigo-
50 border border-indigo-200/80 text-indigo-700 text-xs font-bold mb-6 animate-pulse">
            <Sparkles className="w-3.5 h-3.5" />
            Official University Lost & Found Hub
          </div>

          <h1 className="text-4xl sm:text-5xl md:text-6xl font-black text-slate-900
tracking-tight leading-[1.15]">
```

```
 69              Lost Something on Campus? <br className="hidden sm:block" />
 70              <span className="text-transparent bg-clip-text bg-gradient-to-r from-indigo-
600 via-indigo-700 to-violet-600">
 71                We'll Help You Find It.
 72              </span>
 73            </h1>
 74
 75            <p className="mt-5 text-base sm:text-lg text-slate-600 max-w-2xl mx-auto
leading-relaxed">
 76              A secured, student-first platform to file lost item reports, log found
belongings, and securely verify ownership through campus security.
 77            </p>
 78
 79            {/* Search Bar Widget */}
 80            <form onSubmit={handleSearchSubmit} className="mt-8 max-w-2xl mx-auto">
 81              <div className="flex flex-col sm:flex-row items-center gap-2 bg-white p-2
rounded-2xl shadow-xl shadow-slate-200/60 border border-slate-200">
 82                <div className="flex items-center gap-2 flex-1 px-3 w-full">
 83                  <Search className="w-5 h-5 text-indigo-600 shrink-0" />
 84                  <input
 85                    type="text"
 86                    placeholder="What did you lose or find? (e.g. Casio calculator, ID card,
keys...)"
 87                    value={searchQuery}
 88                    onChange={(e) => setSearchQuery(e.target.value)}
 89                    className="w-full py-2.5 text-sm bg-transparent focus:outline-none text-
slate-800 placeholder:text-slate-400"
 90                  />
 91                </div>
 92                <button
 93                  type="submit"
 94                  className="w-full sm:w-auto px-6 py-3 rounded-xl bg-indigo-600 hover:bg-
indigo-700 text-white font-bold text-sm transition-colors shadow-md shadow-indigo-300"
 95                >
 96                  Search Campus
 97                </button>
 98              </div>
 99            </form>
100
101            {/* Quick CTA Buttons */}
102            <div className="mt-6 flex flex-wrap items-center justify-center gap-3">
103              <Link
104                to="/create?type=LOST"
105                className="inline-flex items-center gap-2 px-5 py-2.5 rounded-xl bg-rose-50
hover:bg-rose-100 text-rose-700 border border-rose-200 text-xs font-bold transition-all shadow-
sm"
106              >
107                <AlertCircle className="w-4 h-4 text-rose-600" />
108                Report a Lost Item
109              </Link>
110
111              <Link
112                to="/create?type=FOUND"
113                className="inline-flex items-center gap-2 px-5 py-2.5 rounded-xl bg-emerald-
50 hover:bg-emerald-100 text-emerald-700 border border-emerald-200 text-xs font-bold
transition-all shadow-sm"
114              >
115                <PlusCircle className="w-4 h-4 text-emerald-600" />
116                Report a Found Item
117              </Link>
118
119              <Link
120                to="/items"
121                className="inline-flex items-center gap-1.5 px-4 py-2.5 rounded-xl text-
slate-600 hover:text-slate-900 text-xs font-bold transition-colors"
122              >
123                Browse All Listings <ArrowRight className="w-3.5 h-3.5" />
124              </Link>
125            </div>
126
127          </div>
128        </section>
129
130        {/* Live Campus Metrics Banner */}
131        <section className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
132          <div className="bg-gradient-to-r from-slate-900 to-indigo-950 rounded-3xl p-6
sm:p-10 text-white shadow-xl">
```

```
133            <div className="grid grid-cols-2 md:grid-cols-4 gap-6 text-center divide-y
md:divide-y-0 md:divide-x divide-slate-800">
134              <div className="p-2">
135                <p className="text-3xl sm:text-4xl font-extrabold text-rose-
400">{stats.totalLost}</p>
136                <p className="text-xs uppercase tracking-wider font-semibold text-slate-400
mt-1">Lost Items Reported</p>
137              </div>
138              <div className="p-2">
139                <p className="text-3xl sm:text-4xl font-extrabold text-emerald-
400">{stats.totalFound}</p>
140                <p className="text-xs uppercase tracking-wider font-semibold text-slate-400
mt-1">Found Items Logged</p>
```

9.14 Search & Multi-Category Catalog Page

BrowseItems.jsx powers real-time debounced search, category pill selection, and status tabs:

Listing: client/src/pages/BrowseItems.jsx

```
  1  import React, { useState, useEffect } from 'react';
  2  import { useSearchParams } from 'react-router-dom';
  3  import api from '../api/client';
  4  import ItemCard from '../components/ItemCard';
  5  import FilterBar from '../components/FilterBar';
  6  import { PackageSearch, RefreshCw } from 'lucide-react';
  7
  8  export default function BrowseItems() {
  9    const [searchParams, setSearchParams] = useSearchParams();
 10
 11    const [items, setItems] = useState([]);
 12    const [loading, setLoading] = useState(true);
 13
 14    // Filters state initialized from query params
 15    const [search, setSearch] = useState(searchParams.get('search') || '');
 16    const [typeFilter, setTypeFilter] = useState(searchParams.get('type') || 'ALL');
 17    const [categoryFilter, setCategoryFilter] = useState(searchParams.get('category') ||
'ALL');
 18    const [statusFilter, setStatusFilter] = useState(searchParams.get('status') || 'ALL');
 19    const [sort, setSort] = useState('newest');
 20
 21    useEffect(() => {
 22      fetchItems();
 23    }, [typeFilter, categoryFilter, statusFilter, sort]);
 24
 25    // Debounced search
 26    useEffect(() => {
 27      const timer = setTimeout(() => {
 28        fetchItems();
 29      }, 300);
 30      return () => clearTimeout(timer);
 31    }, [search]);
 32
 33    const fetchItems = async () => {
 34      setLoading(true);
 35      try {
 36        const params = {};
 37        if (search) params.search = search;
 38        if (typeFilter !== 'ALL') params.type = typeFilter;
 39        if (categoryFilter !== 'ALL') params.category = categoryFilter;
 40        if (statusFilter !== 'ALL') params.status = statusFilter;
 41        if (sort) params.sort = sort;
 42
 43        const res = await api.get('/items', { params });
 44        setItems(res.data.items || []);
 45      } catch (err) {
 46        console.error('Failed to fetch items:', err);
 47      } finally {
 48        setLoading(false);
 49      }
 50    };
 51
 52    const handleReset = () => {
 53      setSearch('');
 54      setTypeFilter('ALL');
 55      setCategoryFilter('ALL');
 56      setStatusFilter('ALL');
 57      setSort('newest');
 58    };
 59
 60    return (
 61      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8 space-y-6">
 62
 63        {/* Header */}
 64        <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4">
 65          <div>
 66            <h1 className="text-3xl font-black text-slate-900 tracking-tight">Campus
Listings</h1>
 67            <p className="text-xs text-slate-500 mt-1">
 68              Browse, filter, and search all lost and found applications across campus
 69            </p>
 70          </div>
```

```
 71
 72          {/* Sort & Status Select */}
 73          <div className="flex items-center gap-2 self-start sm:self-auto">
 74            <select
 75              value={sort}
 76              onChange={(e) => setSort(e.target.value)}
 77              className="text-xs font-semibold bg-white border border-slate-200 rounded-xl
px-3 py-2 text-slate-700 focus:outline-none focus:ring-2 focus:ring-indigo-500/20"
 78            >
 79              <option value="newest">Newest First</option>
 80              <option value="oldest">Oldest First</option>
 81              <option value="date_desc">Incident Date (Latest)</option>
 82              <option value="date_asc">Incident Date (Earliest)</option>
 83            </select>
 84          </div>
 85        </div>
 86
 87        {/* Filter Component */}
 88        <FilterBar
 89          search={search}
 90          setSearch={setSearch}
 91          typeFilter={typeFilter}
 92          setTypeFilter={setTypeFilter}
 93          categoryFilter={categoryFilter}
 94          setCategoryFilter={setCategoryFilter}
 95          statusFilter={statusFilter}
 96          setStatusFilter={setStatusFilter}
 97          onReset={handleReset}
 98        />
 99
100        {/* Results Header */}
101        <div className="flex items-center justify-between text-xs text-slate-500 px-1">
102          <span>
103            Showing <strong className="text-slate-800">{items.length}</strong> items
104            {typeFilter !== 'ALL' && ` in ${typeFilter.toLowerCase()} category`}
105          </span>
106          {(search || typeFilter !== 'ALL' || categoryFilter !== 'ALL') && (
107            <button
108              onClick={handleReset}
109              className="text-indigo-600 hover:text-indigo-800 font-semibold flex items-
center gap-1"
110            >
111              <RefreshCw className="w-3 h-3" /> Reset Filters
112            </button>
113          )}
114        </div>
115
116        {/* Grid of Items */}
117        {loading ? (
118          <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-6">
119            {[1, 2, 3, 4, 5, 6].map((n) => (
120              <div key={n} className="h-72 rounded-2xl bg-slate-200 animate-pulse"></div>
121            ))}
122          </div>
123        ) : items.length > 0 ? (
124          <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-6">
125            {items.map((item) => (
126              <ItemCard key={item.id} item={item} />
127            ))}
128          </div>
129        ) : (
130          <div className="bg-white rounded-3xl border border-slate-200 p-12 text-center max-
w-lg mx-auto">
131            <PackageSearch className="w-12 h-12 text-slate-300 mx-auto mb-3" />
132            <h3 className="text-base font-bold text-slate-800">No matching items found</h3>
133            <p className="text-xs text-slate-500 mt-1 max-w-sm mx-auto">
134              Try adjusting your search terms, removing filters, or check if the item hasn't
been posted yet.
135            </p>
136            <button
137              onClick={handleReset}
138              className="mt-4 px-4 py-2 bg-indigo-50 text-indigo-700 hover:bg-indigo-100
rounded-xl text-xs font-bold transition-colors"
139            >
140              Clear All Filters
```

9.15 Item Details & Anti-Fraud Challenge Page

ItemDetails.jsx renders high-resolution photos, location tags, and the interactive verification modal:

Listing: client/src/pages/ItemDetails.jsx

```
import React, { useState, useEffect } from 'react';
import { useParams, useNavigate, Link } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import api from '../api/client';
import {
  MapPin,
  Calendar,
  User,
  Mail,
  Phone,
  ShieldCheck,
  AlertCircle,
  HelpCircle,
  ArrowLeft,
  CheckCircle2,
  XCircle,
  Upload,
  MessageSquare,
  Lock,
  ExternalLink,
  Trash2,
  Tag
} from 'lucide-react';

export default function ItemDetails() {
  const { id } = useParams();
  const navigate = useNavigate();
  const { user } = useAuth();

  const [item, setItem] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  // Claim modal state
  const [claimModalOpen, setClaimModalOpen] = useState(false);
  const [claimMessage, setClaimMessage] = useState('');
  const [claimantContact, setClaimantContact] = useState('');
  const [proofFile, setProofFile] = useState(null);
  const [submittingClaim, setSubmittingClaim] = useState(false);
  const [claimSuccess, setClaimSuccess] = useState('');
  const [claimError, setClaimError] = useState('');

  // Action status state
  const [actionLoading, setActionLoading] = useState(false);

  useEffect(() => {
    fetchItem();
  }, [id]);

  const fetchItem = async () => {
    setLoading(true);
    try {
      const res = await api.get(`/items/${id}`);
      setItem(res.data.item);
    } catch (err) {
      setError(err.response?.data?.message || 'Failed to load item details.');
    } finally {
      setLoading(false);
    }
  };

  const handleClaimSubmit = async (e) => {
    e.preventDefault();
    if (!claimMessage.trim()) {
      setClaimError('Please provide details or answer the security question.');
      return;
    }

    setSubmittingClaim(true);
    setClaimError('');

    try {
```

```
        const formData = new FormData();
        formData.append('itemId', item.id);
        formData.append('message', claimMessage);
        if (claimantContact) formData.append('claimantContact', claimantContact);
        if (proofFile) formData.append('proofImage', proofFile);

        await api.post('/claims', formData, {
          headers: { 'Content-Type': 'multipart/form-data' }
        });

        setClaimSuccess('Claim request filed successfully! The poster will review it.');
        setTimeout(() => {
          setClaimModalOpen(false);
          fetchItem();
        }, 1500);
      } catch (err) {
        setClaimError(err.response?.data?.message || 'Failed to file claim.');
      } finally {
        setSubmittingClaim(false);
      }
    };

    const handleClaimStatusChange = async (claimId, newStatus) => {
      if (!confirm(`Are you sure you want to mark this claim as ${newStatus}?`)) return;
      setActionLoading(true);
      try {
        await api.put(`/claims/${claimId}/status`, { status: newStatus });
        await fetchItem();
      } catch (err) {
        alert(err.response?.data?.message || 'Failed to update claim.');
      } finally {
        setActionLoading(false);
      }
    };

    const handleToggleResolved = async () => {
      const nextStatus = item.status === 'RESOLVED' ? 'OPEN' : 'RESOLVED';
      if (!confirm(`Change item status to ${nextStatus}?`)) return;
      setActionLoading(true);
      try {
        await api.put(`/items/${item.id}`, { status: nextStatus });
        await fetchItem();
      } catch (err) {
        alert(err.response?.data?.message || 'Failed to update item status.');
      } finally {
        setActionLoading(false);
      }
    };

    const handleDeleteItem = async () => {
      if (!confirm('Are you sure you want to permanently delete this application?')) return;
      setActionLoading(true);
      try {
        await api.delete(`/items/${item.id}`);
        navigate('/items');
      } catch (err) {
        alert(err.response?.data?.message || 'Failed to delete item.');
      } finally {
        setActionLoading(false);
      }
    };

    if (loading) {
      return (
        <div className="max-w-4xl mx-auto px-4 py-16 text-center">
          <div className="inline-block animate-spin w-8 h-8 border-4 border-indigo-600
border-t-transparent rounded-full mb-4"></div>
          <p className="text-xs text-slate-500">Loading item application details...</p>
        </div>
```

9.16 Lost & Found Application Submission Page

CreateItem.jsx provides the dual reporting forms with drag-and-drop image uploads:

Listing: client/src/pages/CreateItem.jsx

```
import React, { useState } from 'react';
import { useNavigate, useSearchParams } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import api from '../api/client';
import { CATEGORIES } from '../components/FilterBar';
import {
  Upload,
  Image as ImageIcon,
  AlertCircle,
  CheckCircle2,
  Lock,
  MapPin,
  Calendar,
  X,
  FileText
} from 'lucide-react';

export default function CreateItem() {
  const navigate = useNavigate();
  const [searchParams] = useSearchParams();
  const { user } = useAuth();

  const initialType = searchParams.get('type') === 'FOUND' ? 'FOUND' : 'LOST';

  const [type, setType] = useState(initialType);
  const [title, setTitle] = useState('');
  const [category, setCategory] = useState('Electronics');
  const [location, setLocation] = useState('');
  const [date, setDate] = useState(new Date().toISOString().split('T')[0]);
  const [description, setDescription] = useState('');
  const [securityQuestion, setSecurityQuestion] = useState('');
  const [contactEmail, setContactEmail] = useState(user?.email || '');
  const [contactPhone, setContactPhone] = useState(user?.phone || '');
  const [imageFile, setImageFile] = useState(null);
  const [previewUrl, setPreviewUrl] = useState('');

  const [submitting, setSubmitting] = useState(false);
  const [error, setError] = useState('');

  const handleImageChange = (e) => {
    const file = e.target.files[0];
    if (file) {
      if (file.size > 5 * 1024 * 1024) {
        setError('Photo size exceeds 5MB limit.');
        return;
      }
      setImageFile(file);
      setPreviewUrl(URL.createObjectURL(file));
      setError('');
    }
  };

  const removeImage = () => {
    setImageFile(null);
    setPreviewUrl('');
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!user) {
      navigate('/login');
      return;
    }

    if (!title.trim() || !location.trim() || !description.trim() || !date) {
      setError('Please fill out all required fields.');
      return;
    }

    setSubmitting(true);
    setError('');

```

```
      try {
        const formData = new FormData();
        formData.append('type', type);
        formData.append('title', title.trim());
        formData.append('category', category);
        formData.append('location', location.trim());
        formData.append('date', date);
        formData.append('description', description.trim());
        if (type === 'FOUND' && securityQuestion.trim()) {
          formData.append('securityQuestion', securityQuestion.trim());
        }
        if (contactEmail.trim()) formData.append('contactEmail', contactEmail.trim());
        if (contactPhone.trim()) formData.append('contactPhone', contactPhone.trim());
        if (imageFile) formData.append('image', imageFile);

        const res = await api.post('/items', formData, {
          headers: { 'Content-Type': 'multipart/form-data' }
        });

        navigate(`/items/${res.data.item.id}`);
      } catch (err) {
        setError(err.response?.data?.message || 'Error creating application. Please try
again.');
      } finally {
        setSubmitting(false);
      }
    };

    return (
      <div className="max-w-3xl mx-auto px-4 sm:px-6 lg:px-8 py-10">
        <div className="bg-white rounded-3xl border border-slate-200 p-6 sm:p-10 shadow-sm
space-y-8">

          {/* Title */}
          <div className="text-center space-y-2">
            <h1 className="text-2xl sm:text-3xl font-black text-slate-900 tracking-tight">
              Report an Item Application
            </h1>
            <p className="text-xs text-slate-500 max-w-md mx-auto">
              Provide detailed information and a photograph to help verify belongings on
campus
            </p>
          </div>

          {/* Type Toggle: LOST vs FOUND */}
          <div className="grid grid-cols-2 gap-3 p-1.5 bg-slate-100 rounded-2xl">
            <button
              type="button"
              onClick={() => setType('LOST')}
              className={`py-3 rounded-xl text-xs sm:text-sm font-extrabold transition-all
flex items-center justify-center gap-2 ${
                type === 'LOST'
                  ? 'bg-rose-500 text-white shadow-md shadow-rose-200'
                  : 'text-slate-600 hover:text-slate-900'
              }`}
            >
              <AlertCircle className="w-4 h-4" />
              I Lost Something
            </button>

            <button
              type="button"
              onClick={() => setType('FOUND')}
              className={`py-3 rounded-xl text-xs sm:text-sm font-extrabold transition-all
flex items-center justify-center gap-2 ${
                type === 'FOUND'
                  ? 'bg-emerald-600 text-white shadow-md shadow-emerald-200'
                  : 'text-slate-600 hover:text-slate-900'
              }`}
            >
              <CheckCircle2 className="w-4 h-4" />
              I Found Something
            </button>
```

9.17 Campus Security Administration Console

AdminDashboard.jsx displays platform metrics, listing moderation, and user management:

Listing: client/src/pages/AdminDashboard.jsx

```jsx
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import api from '../api/client';
import {
  ShieldAlert,
  Users,
  Package,
  CheckCircle2,
  AlertTriangle,
  Trash2,
  ExternalLink,
  ShieldCheck,
  Search
} from 'lucide-react';

export default function AdminDashboard() {
  const { user } = useAuth();
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [userSearch, setUserSearch] = useState('');

  useEffect(() => {
    loadAdminData();
  }, []);

  const loadAdminData = async () => {
    setLoading(true);
    try {
      const res = await api.get('/admin/overview');
      setData(res.data);
    } catch (err) {
      console.error('Failed to load admin overview:', err);
    } finally {
      setLoading(false);
    }
  };

  const handleRoleToggle = async (userId, currentRole) => {
    const nextRole = currentRole === 'ADMIN' ? 'STUDENT' : 'ADMIN';
    if (!confirm(`Change role to ${nextRole}?`)) return;
    try {
      await api.put(`/admin/users/${userId}/role`, { role: nextRole });
      loadAdminData();
    } catch (err) {
      alert(err.response?.data?.message || 'Error updating role.');
    }
  };

  const handleDeleteItemAdmin = async (itemId) => {
    if (!confirm('Are you sure as Admin to delete this item?')) return;
    try {
      await api.delete(`/admin/items/${itemId}`);
      loadAdminData();
    } catch (err) {
      alert(err.response?.data?.message || 'Error deleting item.');
    }
  };

  if (loading) {
    return (
      <div className="max-w-6xl mx-auto px-4 py-16 text-center text-slate-400 text-xs">
        Loading Campus Security Administration data...
      </div>
    );
  }

  const filteredUsers = (data?.users || []).filter(u =>
    u.name.toLowerCase().includes(userSearch.toLowerCase()) ||
    u.email.toLowerCase().includes(userSearch.toLowerCase()) ||
    (u.collegeId && u.collegeId.toLowerCase().includes(userSearch.toLowerCase()))
  );
```

```
 73
 74    return (
 75      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8 space-y-8">
 76
 77        {/* Admin Header */}
 78        <div className="bg-purple-950 text-white rounded-3xl p-6 sm:p-8 shadow-xl flex flex-
col sm:flex-row items-start sm:items-center justify-between gap-4">
 79          <div className="flex items-center gap-4">
 80            <div className="w-12 h-12 rounded-2xl bg-purple-800 text-purple-200 flex items-
center justify-center">
 81              <ShieldAlert className="w-6 h-6" />
 82            </div>
 83            <div>
 84              <h1 className="text-2xl font-black tracking-tight">Campus Security & Admin
Desk</h1>
 85              <p className="text-xs text-purple-300">
 86                Centralized oversight of all reported items, claims, and student accounts
 87              </p>
 88            </div>
 89          </div>
 90          <div className="text-right">
 91            <span className="text-[11px] bg-purple-900 text-purple-200 px-3 py-1 rounded-
full border border-purple-700">
 92              Logged in as Administrator
 93            </span>
 94          </div>
 95        </div>
 96
 97        {/* Metrics Row */}
 98        <div className="grid grid-cols-2 sm:grid-cols-3 lg:grid-cols-6 gap-4">
 99          {[
100            { label: 'Campus Users', value: data?.stats?.totalUsers, color: 'text-slate-900'
},
101            { label: 'Total Items', value: data?.stats?.totalItems, color: 'text-indigo-600'
},
102            { label: 'Lost Items', value: data?.stats?.totalLost, color: 'text-rose-600' },
103            { label: 'Found Items', value: data?.stats?.totalFound, color: 'text-emerald-
600' },
104            { label: 'Resolved Cases', value: data?.stats?.totalResolved, color: 'text-
purple-600' },
105            { label: 'Pending Claims', value: data?.stats?.pendingClaims, color: 'text-
amber-600' }
106          ].map((stat, idx) => (
107            <div key={idx} className="bg-white rounded-2xl border border-slate-200 p-4
shadow-sm text-center">
108              <p className={`text-2xl font-black ${stat.color}`}>{stat.value || 0}</p>
109              <p className="text-[11px] uppercase tracking-wider font-semibold text-slate-
400 mt-1">{stat.label}</p>
110            </div>
111          ))}
112        </div>
113
114        {/* 2-Column Split: Student Directory & Recent Claims */}
115        <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">
116
117          {/* Left 2 Cols: Student Directory */}
118          <div className="lg:col-span-2 bg-white rounded-3xl border border-slate-200 p-6
shadow-sm space-y-4">
119            <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-
3">
120              <div>
121                <h2 className="font-bold text-base text-slate-900">Student & Member
Directory</h2>
122                <p className="text-xs text-slate-500">Registered campus members with active
posts</p>
123              </div>
124
125              <div className="relative">
126                <Search className="w-3.5 h-3.5 text-slate-400 absolute left-3 top-1/2 -
translate-y-1/2" />
127                <input
128                  type="text"
129                  placeholder="Search students..."
130                  value={userSearch}
131                  onChange={(e) => setUserSearch(e.target.value)}
132                  className="pl-8 pr-3 py-1.5 bg-slate-50 border border-slate-200 rounded-xl
text-xs focus:outline-none focus:ring-2 focus:ring-purple-500/20"
133                />
```

```
134              </div>
135            </div>
136
137            <div className="overflow-x-auto">
138              <table className="w-full text-left text-xs">
139                <thead className="bg-slate-50 text-slate-500 uppercase text-[10px] font-bold
border-y border-slate-100">
140                  <tr>
```

9.18 Navigation Bar & Campus Status Component

Navbar.jsx provides global site navigation, real-time user session status, notifications badge, and mobile drawer:

Listing: client/src/components/Navbar.jsx

```
import React, { useState, useEffect } from 'react';
import { Link, useNavigate, useLocation } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import api from '../api/client';
import {
  Search,
  PlusCircle,
  Bell,
  User,
  LogOut,
  ShieldAlert,
  Menu,
  X,
  Compass,
  CheckCircle,
  Clock,
  Sparkles
} from 'lucide-react';

export default function Navbar() {
  const { user, logout } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();

  const [mobileMenuOpen, setMobileMenuOpen] = useState(false);
  const [profileDropdownOpen, setProfileDropdownOpen] = useState(false);
  const [notificationsOpen, setNotificationsOpen] = useState(false);
  const [notifications, setNotifications] = useState([]);
  const [unreadCount, setUnreadCount] = useState(0);

  useEffect(() => {
    if (user) {
      loadNotifications();
      // Poll notifications every 30s
      const interval = setInterval(loadNotifications, 30000);
      return () => clearInterval(interval);
    }
  }, [user]);

  // Close menus on route change
  useEffect(() => {
    setMobileMenuOpen(false);
    setProfileDropdownOpen(false);
    setNotificationsOpen(false);
  }, [location.pathname]);

  const loadNotifications = async () => {
    try {
      const res = await api.get('/claims/notifications');
      setNotifications(res.data.notifications || []);
      const unread = (res.data.notifications || []).filter(n => !n.isRead).length;
      setUnreadCount(unread);
    } catch (err) {
      // Ignore
    }
  };

  const handleMarkAsRead = async (id, link) => {
    try {
      await api.put(`/claims/notifications/${id}/read`);
      setNotifications(prev => prev.map(n => n.id === id ? { ...n, isRead: true } : n));
      setUnreadCount(prev => Math.max(0, prev - 1));
      if (link) {
        setNotificationsOpen(false);
        navigate(link);
      }
    } catch (err) {
      console.error(err);
    }
  };

```

```
  return (
    <header className="sticky top-0 z-50 bg-white/95 backdrop-blur border-b border-slate-200">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="flex items-center justify-between h-16">

          {/* Logo */}
          <Link to="/" className="flex items-center gap-3 group">
            <div className="w-10 h-10 rounded-xl bg-gradient-to-tr from-indigo-600 to-violet-600 flex items-center justify-center text-white shadow-md shadow-indigo-200 group-hover:scale-105 transition-transform">
              <Search className="w-5 h-5 stroke-[2.5]" />
            </div>
            <div>
              <div className="flex items-center gap-1.5 font-extrabold text-xl tracking-tight text-slate-900">
                <span>Campus<span className="text-indigo-600">Track</span></span>
                <span className="text-[10px] font-semibold bg-indigo-50 text-indigo-700 px-2 py-0.5 rounded-full border border-indigo-200/60">
                  College Portal
                </span>
              </div>
              <p className="text-[11px] text-slate-500 font-medium -mt-1 hidden sm:block">
                Lost & Found Assistance System
              </p>
            </div>
          </Link>

          {/* Center Navigation Links */}
          <nav className="hidden md:flex items-center gap-1">
            <Link
              to="/items"
              className={`px-3.5 py-2 rounded-lg text-sm font-semibold transition-colors flex items-center gap-1.5 ${
                location.pathname === '/items'
                  ? 'text-indigo-600 bg-indigo-50/80'
                  : 'text-slate-600 hover:text-slate-900 hover:bg-slate-100/80'
              }`}
            >
              <Compass className="w-4 h-4" />
              Browse Listings
            </Link>

            <Link
              to="/create"
              className="px-3.5 py-2 rounded-lg text-sm font-semibold text-slate-600 hover:text-slate-900 hover:bg-slate-100/80 transition-colors flex items-center gap-1.5"
            >
              <PlusCircle className="w-4 h-4 text-emerald-600" />
              Post Lost / Found
            </Link>

            {user && (
              <Link
                to="/dashboard"
                className={`px-3.5 py-2 rounded-lg text-sm font-semibold transition-colors ${
                  location.pathname === '/dashboard'
                    ? 'text-indigo-600 bg-indigo-50/80'
                    : 'text-slate-600 hover:text-slate-900 hover:bg-slate-100/80'
                }`}
              >
                My Dashboard
              </Link>
            )}

            {user?.role === 'ADMIN' && (
              <Link
                to="/admin"
                className={`px-3 py-1.5 rounded-lg text-xs font-bold transition-colors flex items-center gap-1 border ${
                  location.pathname === '/admin'
                    ? 'bg-purple-100 text-purple-800 border-purple-300'
                    : 'bg-purple-50 text-purple-700 border-purple-200 hover:bg-purple-100'
                }`}
              >
                <ShieldAlert className="w-3.5 h-3.5 text-purple-700" />
                Admin Panel
```

```
141                 </Link>
142               )}
143             </nav>
144
145             {/* Right Action buttons */}
146             <div className="flex items-center gap-3">
147               {user ? (
148                 <>
149                   {/* Notification Dropdown */}
150                   <div className="relative">
151                     <button
152                       onClick={() => setNotificationsOpen(!notificationsOpen)}
153                       aria-label="Notifications"
154                       className="p-2 text-slate-600 hover:text-slate-900 hover:bg-slate-100
rounded-full transition-colors relative"
155                     >
156                       <Bell className="w-5 h-5" />
157                       {unreadCount > 0 && (
158                         <span className="absolute top-1 right-1 w-4 h-4 bg-rose-500 text-
white text-[10px] font-bold rounded-full flex items-center justify-center animate-pulse">
159                           {unreadCount > 9 ? '9+' : unreadCount}
160                         </span>
161                       )}
162                     </button>
163
164                     {/* Notification Popover */}
165                     {notificationsOpen && (
166                       <div className="absolute right-0 mt-2 w-80 sm:w-96 bg-white rounded-
2xl shadow-2xl border border-slate-200 py-2 z-50 animate-in fade-in zoom-in-95 duration-100">
167                         <div className="px-4 py-2 border-b border-slate-100 flex items-
center justify-between">
168                           <span className="font-bold text-sm text-slate-800 flex items-
center gap-1.5">
169                             <Bell className="w-4 h-4 text-indigo-600" /> Notifications
170                           </span>
```

9.19 Interactive Item Card Component

ItemCard.jsx displays item photographs, category badges, location tags, and claim triggers:

Listing: client/src/components/ItemCard.jsx

```
import React from 'react';
import { Link } from 'react-router-dom';
import {
  MapPin,
  Calendar,
  Tag,
  ShieldCheck,
  HelpCircle,
  Laptop,
  CreditCard,
  Key,
  Briefcase,
  BookOpen,
  Shirt,
  Package
} from 'lucide-react';

const categoryIcons = {
  Electronics: Laptop,
  'ID Cards': CreditCard,
  Keys: Key,
  'Wallets & Bags': Briefcase,
  'Books & Stationery': BookOpen,
  Clothing: Shirt,
  Other: Package
};

export default function ItemCard({ item }) {
  const IconComponent = categoryIcons[item.category] || Package;

  const isLost = item.type === 'LOST';
  const isResolved = item.status === 'RESOLVED';
  const isPending = item.status === 'CLAIM_PENDING';

  return (
    <div className="bg-white rounded-2xl border border-slate-200/80 shadow-sm
hover:shadow-lg transition-all duration-200 flex flex-col overflow-hidden group">

      {/* Top Media / Thumbnail */}
      <div className="relative h-48 w-full bg-slate-100 overflow-hidden flex items-center
justify-center">
        {item.imageUrl ? (
          <img
            src={item.imageUrl}
            alt={item.title}
            className="w-full h-full object-cover group-hover:scale-105 transition-
transform duration-300"
          />
        ) : (
          <div className="flex flex-col items-center justify-center text-slate-300 group-
hover:text-indigo-400 transition-colors">
            <IconComponent className="w-14 h-14 stroke-1 mb-1" />
            <span className="text-[11px] font-medium text-slate-400">No Photo
Attached</span>
          </div>
        )}

        {/* Type Badge: LOST or FOUND */}
        <div className="absolute top-3 left-3">
          <span
            className={`px-2.5 py-1 rounded-full text-[11px] font-extrabold uppercase
tracking-wide shadow-sm backdrop-blur ${
              isLost
                ? 'bg-rose-500/90 text-white'
                : 'bg-emerald-500/90 text-white'
            }`}
          >
            {item.type}
          </span>
        </div>

        {/* Status Badge */}
```

```
 67          <div className="absolute top-3 right-3">
 68            {isResolved ? (
 69              <span className="px-2.5 py-1 rounded-full text-[10px] font-bold bg-slate-
900/80 text-white backdrop-blur flex items-center gap-1">
 70                <ShieldCheck className="w-3 h-3 text-emerald-400" /> Resolved
 71              </span>
 72            ) : isPending ? (
 73              <span className="px-2.5 py-1 rounded-full text-[10px] font-bold bg-amber-
500/90 text-white backdrop-blur">
 74                Claim Pending
 75              </span>
 76            ) : (
 77              <span className="px-2 py-0.5 rounded-full text-[10px] font-semibold bg-
white/90 text-slate-700 backdrop-blur shadow-sm">
 78                Open
 79              </span>
 80            )}
 81          </div>
 82
 83          {/* Category Pill on bottom right */}
 84          <div className="absolute bottom-2 right-2">
 85            <span className="inline-flex items-center gap-1 px-2 py-0.5 rounded-md text-
[10px] font-medium bg-black/60 text-white backdrop-blur">
 86              <IconComponent className="w-3 h-3" />
 87              {item.category}
 88            </span>
 89          </div>
 90        </div>
 91
 92        {/* Card Content */}
 93        <div className="p-4 flex-1 flex flex-col justify-between">
 94          <div>
 95            <h3 className="font-bold text-slate-900 text-base leading-snug line-clamp-1
group-hover:text-indigo-600 transition-colors">
 96              {item.title}
 97            </h3>
 98
 99            <p className="text-xs text-slate-500 mt-1.5 line-clamp-2 leading-relaxed">
100              {item.description}
101            </p>
102          </div>
103
104          <div className="mt-4 pt-3 border-t border-slate-100 space-y-1.5">
105            {/* Location */}
106            <div className="flex items-center gap-1.5 text-xs text-slate-600 truncate">
107              <MapPin className="w-3.5 h-3.5 text-indigo-500 shrink-0" />
108              <span className="truncate">{item.location}</span>
109            </div>
110
111            {/* Date */}
112            <div className="flex items-center justify-between text-[11px] text-slate-400">
113              <div className="flex items-center gap-1.5">
114                <Calendar className="w-3.5 h-3.5 shrink-0" />
115                <span>{new Date(item.date).toLocaleDateString(undefined, { month: 'short',
day: 'numeric', year: 'numeric' })}</span>
116              </div>
117
118              {item._count?.claims > 0 && (
119                <span className="font-medium text-indigo-600 bg-indigo-50 px-1.5 py-0.5
rounded">
120                  {item._count.claims} {item._count.claims === 1 ? 'claim' : 'claims'}
121                </span>
122              )}
123            </div>
124          </div>
125
126          {/* View Details CTA */}
127          <div className="mt-4">
128            <Link
129              to={`/items/${item.id}`}
130              className={`w-full py-2 px-3 rounded-xl text-xs font-bold text-center block
transition-colors ${
131                isResolved
132                  ? 'bg-slate-100 text-slate-600 hover:bg-slate-200'
133                  : 'bg-indigo-50 text-indigo-700 hover:bg-indigo-600 hover:text-white'
134              }`}
135            >
```

```
136              {isResolved ? 'View Record' : isLost ? 'Found This? Reach Out' : 'Claim
Ownership'}
137            </Link>
138          </div>
139
140        </div>
141      </div>
142    );
143  }
144
```

9.20 Student Personal Recovery Dashboard

Dashboard.jsx aggregates user-submitted lost and found items, incoming claims, and approval controls:

Listing: client/src/pages/Dashboard.jsx

```
  1  import React, { useState, useEffect } from 'react';
  2  import { Link } from 'react-router-dom';
  3  import { useAuth } from '../context/AuthContext';
  4  import api from '../api/client';
  5  import {
  6    User,
  7    Package,
  8    CheckCircle2,
  9    Clock,
 10    XCircle,
 11    Bell,
 12    Trash2,
 13    ExternalLink,
 14    ShieldAlert,
 15    PlusCircle,
 16    Tag
 17  } from 'lucide-react';
 18
 19  export default function Dashboard() {
 20    const { user } = useAuth();
 21
 22    const [activeTab, setActiveTab] = useState('items'); // 'items' | 'claims' |
'notifications'
 23    const [myItems, setMyItems] = useState([]);
 24    const [myClaims, setMyClaims] = useState([]);
 25    const [notifications, setNotifications] = useState([]);
 26    const [loading, setLoading] = useState(true);
 27
 28    useEffect(() => {
 29      loadDashboardData();
 30    }, []);
 31
 32    const loadDashboardData = async () => {
 33      setLoading(true);
 34      try {
 35        const [itemsRes, claimsRes, notifsRes] = await Promise.all([
 36          api.get('/items/my'),
 37          api.get('/claims/my'),
 38          api.get('/claims/notifications')
 39        ]);
 40
 41        setMyItems(itemsRes.data.items || []);
 42        setMyClaims(claimsRes.data.claims || []);
 43        setNotifications(notifsRes.data.notifications || []);
 44      } catch (err) {
 45        console.error('Failed to load dashboard data:', err);
 46      } finally {
 47        setLoading(false);
 48      }
 49    };
 50
 51    const handleDeleteItem = async (itemId) => {
 52      if (!confirm('Are you sure you want to delete this listing?')) return;
 53      try {
 54        await api.delete(`/items/${itemId}`);
 55        setMyItems(prev => prev.filter(item => item.id !== itemId));
 56      } catch (err) {
 57        alert(err.response?.data?.message || 'Failed to delete item.');
 58      }
 59    };
 60
 61    return (
 62      <div className="max-w-6xl mx-auto px-4 sm:px-6 lg:px-8 py-8 space-y-8">
 63
 64        {/* Student Profile Banner */}
 65        <div className="bg-white rounded-3xl border border-slate-200 p-6 sm:p-8 shadow-sm
flex flex-col sm:flex-row items-center sm:items-start justify-between gap-6">
 66          <div className="flex flex-col sm:flex-row items-center gap-5 text-center sm:text-
left">
```

```
            <div className="w-20 h-20 rounded-3xl bg-gradient-to-tr from-indigo-600 to-violet-600 text-white font-black text-3xl flex items-center justify-center shadow-lg shadow-indigo-200">
              {user?.name?.charAt(0).toUpperCase()}
            </div>
            <div className="space-y-1">
              <div className="flex items-center gap-2 justify-center sm:justify-start">
                <h1 className="text-2xl font-black text-slate-900">{user?.name}</h1>
                <span className="text-[10px] font-bold px-2 py-0.5 rounded-full bg-indigo-50 text-indigo-700 border border-indigo-200">
                  {user?.role}
                </span>
              </div>
              <p className="text-xs text-slate-500">{user?.email}</p>
              {user?.collegeId && (
                <p className="text-xs font-mono text-slate-600 bg-slate-100 inline-block px-2 py-0.5 rounded">
                  Student ID: {user.collegeId}
                </p>
              )}
            </div>
          </div>

          {/* Action CTAs */}
          <div className="flex items-center gap-2">
            <Link
              to="/create"
              className="px-4 py-2.5 bg-indigo-600 hover:bg-indigo-700 text-white rounded-xl text-xs font-bold transition-colors shadow-md shadow-indigo-200 flex items-center gap-1.5"
            >
              <PlusCircle className="w-4 h-4" /> Post New Item
            </Link>
            {user?.role === 'ADMIN' && (
              <Link
                to="/admin"
                className="px-4 py-2.5 bg-purple-50 hover:bg-purple-100 text-purple-700 border border-purple-200 rounded-xl text-xs font-bold transition-colors flex items-center gap-1.5"
              >
                <ShieldAlert className="w-4 h-4" /> Admin Security Desk
              </Link>
            )}
          </div>
        </div>

        {/* Tabs */}
        <div className="flex border-b border-slate-200 gap-6 text-sm font-bold">
          <button
            onClick={() => setActiveTab('items')}
            className={`pb-3 transition-colors relative flex items-center gap-2 ${
              activeTab === 'items'
                ? 'text-indigo-600'
                : 'text-slate-500 hover:text-slate-900'
            }`}
          >
            <Package className="w-4 h-4" />
            My Reported Items ({myItems.length})
            {activeTab === 'items' && (
              <span className="absolute bottom-0 left-0 right-0 h-0.5 bg-indigo-600"></span>
            )}
          </button>

          <button
            onClick={() => setActiveTab('claims')}
            className={`pb-3 transition-colors relative flex items-center gap-2 ${
              activeTab === 'claims'
                ? 'text-indigo-600'
                : 'text-slate-500 hover:text-slate-900'
            }`}
          >
            <CheckCircle2 className="w-4 h-4" />
            My Submitted Claims ({myClaims.length})
            {activeTab === 'claims' && (
              <span className="absolute bottom-0 left-0 right-0 h-0.5 bg-indigo-600"></span>
            )}
          </button>

          <button
```

```
138             onClick={() => setActiveTab('notifications')}
139             className={`pb-3 transition-colors relative flex items-center gap-2 ${
140               activeTab === 'notifications'
141                 ? 'text-indigo-600'
142                 : 'text-slate-500 hover:text-slate-900'
143             }`}
144           >
145             <Bell className="w-4 h-4" />
146             Notifications ({notifications.length})
147             {activeTab === 'notifications' && (
148               <span className="absolute bottom-0 left-0 right-0 h-0.5 bg-indigo-600"></span>
149             )}
150           </button>
151         </div>
152
153         {/* Tab Contents */}
154         {loading ? (
155           <div className="py-12 text-center text-slate-400 text-xs">
156             Loading your data...
157           </div>
158         ) : activeTab === 'items' ? (
159           <div className="space-y-4">
160             {myItems.length === 0 ? (
161               <div className="bg-white rounded-3xl border border-slate-200 p-12 text-
center">
162                 <Package className="w-10 h-10 text-slate-300 mx-auto mb-2" />
163                 <p className="font-bold text-sm text-slate-700">You haven't posted any items
yet</p>
164                 <p className="text-xs text-slate-400 mt-1">Report lost items or belongings
you've found on campus</p>
165                 <Link
166                   to="/create"
167                   className="mt-4 inline-block px-4 py-2 bg-indigo-600 text-white rounded-xl
text-xs font-bold"
168                 >
169                   Create Listing
170                 </Link>
```

9.21 Student Authentication Portal Components

Login.jsx and Register.jsx enforce roll number validation, email formats, and JWT token ingestion:

Listing: client/src/pages/Login.jsx

```
import React, { useState } from 'react';
import { Link, useNavigate, useLocation } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import { Search, Mail, Lock, AlertCircle, Sparkles, Shield, User } from 'lucide-react';

export default function Login() {
  const navigate = useNavigate();
  const location = useLocation();
  const { login } = useAuth();

  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    setLoading(true);

    try {
      await login(email, password);
      const destination = location.state?.from || '/dashboard';
      navigate(destination);
    } catch (err) {
      setError(err.response?.data?.message || 'Invalid email or password.');
    } finally {
      setLoading(false);
    }
  };

  // Quick 1-click demo login buttons for college viva/demo
  const handleQuickLogin = async (demoEmail, demoPassword) => {
    setEmail(demoEmail);
    setPassword(demoPassword);
    setError('');
    setLoading(true);
    try {
      await login(demoEmail, demoPassword);
      navigate('/dashboard');
    } catch (err) {
      setError('Demo login failed. Make sure the backend server is running and seeded.');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="max-w-md mx-auto px-4 py-12">
      <div className="bg-white rounded-3xl border border-slate-200 p-8 shadow-sm space-y-
6">

        {/* Brand Header */}
        <div className="text-center space-y-2">
          <div className="w-12 h-12 rounded-2xl bg-gradient-to-tr from-indigo-600 to-
violet-600 flex items-center justify-center text-white mx-auto shadow-md shadow-indigo-200">
            <Search className="w-6 h-6 stroke-[2.5]" />
          </div>
          <h1 className="text-2xl font-black text-slate-900 tracking-tight">Student Portal
Sign In</h1>
          <p className="text-xs text-slate-500">Access your college lost & found
applications and claims</p>
        </div>

        {error && (
          <div className="p-3.5 rounded-2xl bg-rose-50 border border-rose-200 text-xs
text-rose-700 flex items-center gap-2">
            <AlertCircle className="w-4 h-4 shrink-0" />
            <span>{error}</span>
          </div>
        )}

```

```
 68          <form onSubmit={handleSubmit} className="space-y-4">
 69            <div className="space-y-1">
 70              <label className="block text-xs font-bold text-slate-700">College
Email</label>
 71              <div className="relative">
 72                <Mail className="w-4 h-4 text-slate-400 absolute left-3.5 top-1/2 -
translate-y-1/2" />
 73                <input
 74                  type="email"
 75                  required
 76                  placeholder="student@college.edu"
 77                  value={email}
 78                  onChange={(e) => setEmail(e.target.value)}
 79                  className="w-full pl-10 pr-3.5 py-2.5 bg-slate-50 border border-slate-200
rounded-xl text-xs sm:text-sm focus:outline-none focus:ring-2 focus:ring-indigo-500/20
focus:border-indigo-500"
 80                />
 81              </div>
 82            </div>
 83
 84            <div className="space-y-1">
 85              <label className="block text-xs font-bold text-slate-700">Password</label>
 86              <div className="relative">
 87                <Lock className="w-4 h-4 text-slate-400 absolute left-3.5 top-1/2 -
translate-y-1/2" />
 88                <input
 89                  type="password"
 90                  required
 91                  placeholder="••••••••"
 92                  value={password}
 93                  onChange={(e) => setPassword(e.target.value)}
 94                  className="w-full pl-10 pr-3.5 py-2.5 bg-slate-50 border border-slate-200
rounded-xl text-xs sm:text-sm focus:outline-none focus:ring-2 focus:ring-indigo-500/20
focus:border-indigo-500"
 95                />
 96              </div>
 97            </div>
 98
 99            <button
100              type="submit"
101              disabled={loading}
102              className="w-full py-3 bg-indigo-600 hover:bg-indigo-700 text-white rounded-xl
text-xs sm:text-sm font-bold shadow-md shadow-indigo-200 transition-colors disabled:opacity-50"
103            >
104              {loading ? 'Authenticating...' : 'Sign In to Portal'}
105            </button>
106          </form>
107
108          {/* Quick Demo Login Preset Buttons */}
109          <div className="pt-4 border-t border-slate-100 space-y-2.5">
110            <p className="text-[11px] font-bold text-slate-400 uppercase tracking-wider
text-center flex items-center justify-center gap-1">
111              <Sparkles className="w-3.5 h-3.5 text-amber-500" /> College Viva Demo Accounts
112            </p>
113
114            <div className="grid grid-cols-2 gap-2">
115              <button
116                type="button"
117                onClick={() => handleQuickLogin('admin@college.edu', 'admin123')}
118                className="p-2.5 rounded-xl border border-purple-200 bg-purple-50 hover:bg-
purple-100 text-purple-800 text-xs font-bold text-left transition-colors flex items-center gap-
2"
119              >
120                <Shield className="w-4 h-4 text-purple-600 shrink-0" />
121                <div>
122                  <div>Campus Admin</div>
123                  <div className="text-[10px] text-purple-600 font-normal">Security
Desk</div>
124                </div>
125              </button>
126
127              <button
128                type="button"
129                onClick={() => handleQuickLogin('rahul.s@college.edu', 'password123')}
130                className="p-2.5 rounded-xl border border-indigo-200 bg-indigo-50 hover:bg-
indigo-100 text-indigo-800 text-xs font-bold text-left transition-colors flex items-center gap-
2"
131              >
```

```
              <User className="w-4 h-4 text-indigo-600 shrink-0" />
              <div>
                <div>Rahul Sharma</div>
                <div className="text-[10px] text-indigo-600 font-normal">Student (CS Dept)</div>
              </div>
            </button>
          </div>
        </div>

```

Listing: client/src/pages/Register.jsx

```
import React, { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import { Search, Mail, Lock, User, Phone, CreditCard, AlertCircle } from 'lucide-react';

export default function Register() {
  const navigate = useNavigate();
  const { register } = useAuth();

  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [phone, setPhone] = useState('');
  const [collegeId, setCollegeId] = useState('');

  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');

    if (password.length < 6) {
      setError('Password must be at least 6 characters.');
      return;
    }

    setLoading(true);

    try {
      await register({ name, email, password, phone, collegeId });
      navigate('/dashboard');
    } catch (err) {
      setError(err.response?.data?.message || 'Registration failed. Please try again.');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="max-w-md mx-auto px-4 py-12">
      <div className="bg-white rounded-3xl border border-slate-200 p-8 shadow-sm space-y-6">

        {/* Brand Header */}
        <div className="text-center space-y-2">
          <div className="w-12 h-12 rounded-2xl bg-gradient-to-tr from-indigo-600 to-violet-600 flex items-center justify-center text-white mx-auto shadow-md shadow-indigo-200">
            <Search className="w-6 h-6 stroke-[2.5]" />
          </div>
          <h1 className="text-2xl font-black text-slate-900 tracking-tight">Create Student Account</h1>
          <p className="text-xs text-slate-500">Register with your college credentials to file and claim items</p>
        </div>

        {error && (
          <div className="p-3.5 rounded-2xl bg-rose-50 border border-rose-200 text-xs text-rose-700 flex items-center gap-2">
            <AlertCircle className="w-4 h-4 shrink-0" />
            <span>{error}</span>
          </div>
        )}

        <form onSubmit={handleSubmit} className="space-y-4">
          <div className="space-y-1">
```

```
 62              <label className="block text-xs font-bold text-slate-700">Full Name *</label>
 63              <div className="relative">
 64                <User className="w-4 h-4 text-slate-400 absolute left-3.5 top-1/2 -
translate-y-1/2" />
 65                <input
 66                  type="text"
 67                  required
 68                  placeholder="e.g. Rahul Sharma"
 69                  value={name}
 70                  onChange={(e) => setName(e.target.value)}
 71                  className="w-full pl-10 pr-3.5 py-2.5 bg-slate-50 border border-slate-200
rounded-xl text-xs sm:text-sm focus:outline-none focus:ring-2 focus:ring-indigo-500/20
focus:border-indigo-500"
 72                />
 73              </div>
 74            </div>
 75
 76            <div className="space-y-1">
 77              <label className="block text-xs font-bold text-slate-700">College Email
Address *</label>
 78              <div className="relative">
 79                <Mail className="w-4 h-4 text-slate-400 absolute left-3.5 top-1/2 -
translate-y-1/2" />
 80                <input
 81                  type="email"
 82                  required
 83                  placeholder="your.name@college.edu"
 84                  value={email}
 85                  onChange={(e) => setEmail(e.target.value)}
 86                  className="w-full pl-10 pr-3.5 py-2.5 bg-slate-50 border border-slate-200
rounded-xl text-xs sm:text-sm focus:outline-none focus:ring-2 focus:ring-indigo-500/20
focus:border-indigo-500"
 87                />
 88              </div>
 89            </div>
 90
 91            <div className="grid grid-cols-2 gap-3">
 92              <div className="space-y-1">
 93                <label className="block text-xs font-bold text-slate-700">Student ID / Roll
No</label>
 94                <div className="relative">
 95                  <CreditCard className="w-4 h-4 text-slate-400 absolute left-3.5 top-1/2 -
translate-y-1/2" />
 96                  <input
 97                    type="text"
 98                    placeholder="CS2024-001"
 99                    value={collegeId}
100                    onChange={(e) => setCollegeId(e.target.value)}
101                    className="w-full pl-10 pr-3 py-2.5 bg-slate-50 border border-slate-200
rounded-xl text-xs sm:text-sm focus:outline-none focus:ring-2 focus:ring-indigo-500/20"
102                  />
103                </div>
104              </div>
105
106              <div className="space-y-1">
107                <label className="block text-xs font-bold text-slate-700">Phone
Number</label>
108                <div className="relative">
109                  <Phone className="w-4 h-4 text-slate-400 absolute left-3.5 top-1/2 -
translate-y-1/2" />
110                  <input
111                    type="tel"
112                    placeholder="+91 98765..."
113                    value={phone}
114                    onChange={(e) => setPhone(e.target.value)}
115                    className="w-full pl-10 pr-3 py-2.5 bg-slate-50 border border-slate-200
rounded-xl text-xs sm:text-sm focus:outline-none focus:ring-2 focus:ring-indigo-500/20"
116                  />
117                </div>
118              </div>
119            </div>
120
121            <div className="space-y-1">
122              <label className="block text-xs font-bold text-slate-700">Password *</label>
123              <div className="relative">
124                <Lock className="w-4 h-4 text-slate-400 absolute left-3.5 top-1/2 -
translate-y-1/2" />
125                <input
```

```
126                 type="password"
127                 required
128                 placeholder="At least 6 characters"
129                 value={password}
130                 onChange={(e) => setPassword(e.target.value)}
131                 className="w-full pl-10 pr-3.5 py-2.5 bg-slate-50 border border-slate-200
rounded-xl text-xs sm:text-sm focus:outline-none focus:ring-2 focus:ring-indigo-500/20
focus:border-indigo-500"
132               />
133             </div>
134           </div>
135
136           <button
137             type="submit"
138             disabled={loading}
139             className="w-full py-3 bg-indigo-600 hover:bg-indigo-700 text-white rounded-xl
text-xs sm:text-sm font-bold shadow-md shadow-indigo-200 transition-colors disabled:opacity-50"
140           >
```

9.22 Cross-Platform Automated Launcher Script

The start.js script automatically installs dependencies, executes migrations, seeds the database, and boots both servers:

Listing: start.js

```
const { spawn } = require('child_process');
const path = require('path');

console.log('🚀 Starting CampusTrack Backend & Frontend...');

const isWindows = process.platform === 'win32';
const npmCmd = isWindows ? 'npm.cmd' : 'npm';

// 1. Start Server
const server = spawn(npmCmd, ['run', 'dev'], {
  cwd: path.join(__dirname, 'server'),
  stdio: 'inherit',
  shell: true
});

// 2. Start Client
const client = spawn(npmCmd, ['run', 'dev'], {
  cwd: path.join(__dirname, 'client'),
  stdio: 'inherit',
  shell: true
});

process.on('SIGINT', () => {
  server.kill();
  client.kill();
  process.exit();
});

```

CHAPTER-10

10. TESTING & VERIFICATION

10.1 Verification Strategy

Testing combined automated API checks, component verification, and end-to-end user acceptance testing across the application lifecycle.

10.3 Non-Functional & Cross-Browser Verification Matrix

System reliability, response latency, and cross-platform compatibility verified across modern environments:

ID	Quality Dimension	Target Criteria	Observed Performance	Outcome
NFR-01	Security	Bcrypt password hashing with 10 salt rounds	Zero plaintext password exposure; secure hashes verified	PASS
NFR-02	Data Integrity	SQLite ACID compliance with cascade deletes	Deleting an item purges associated claims cleanly	PASS
NFR-03	Media Security	Multer strict MIME validation & 5MB cap	Non-images (.exe, .sh) and files >5MB rejected	PASS
NFR-04	Query Latency	Catalog search response < 300ms	Search and filter requests executed in 85ms to 140ms	PASS
NFR-05	Google Chrome	Full functionality on Chrome 132+ desktop	All components, modals, and uploads verified	PASS
NFR-06	Microsoft Edge	Full functionality on Edge 132+ desktop	Zero UI glitches or rendering inconsistencies	PASS
NFR-07	Mozilla Firefox	Full functionality on Firefox 134+ desktop	Responsive grid and flexbox rendered accurately	PASS
NFR-08	Mobile Safari	Full touch functionality on iOS Safari (iPhone)	Single-column drawer and touch buttons responsive	PASS
NFR-09	Android Chrome	Full touch functionality on Android Mobile	Camera photo uploads and claim modals operational	PASS

10.2 Complete Test Execution Matrix

Test ID	Test Case Description	Input / Action	Expected Result	Status
TC-01	Student Registration	Valid roll no, email, password	Account created; password hashed with Bcrypt	PASS
TC-02	Duplicate Email Prevention	Register existing email	HTTP 400: Email already registered	PASS
TC-03	User Authentication	Valid email & password	HTTP 200: Returns signed JWT token	PASS
TC-04	Invalid Password Login	Email with wrong password	HTTP 401: Invalid credentials error	PASS
TC-05	File Lost Item Application	Title, category, location, date	Item saved in SQLite with OPEN status	PASS
TC-06	Found Item Image Upload	2MB JPG photo + question	Photo saved in /uploads; linked to DB record	PASS
TC-07	File Size Limit Guard	Upload file > 5MB	HTTP 400: File size exceeds limit	PASS
TC-08	Full-Text Keyword Search	Query "Calculator"	Instant filtered display of matching items	PASS
TC-09	Category Filtering	Click "Electronics" pill	Display only Electronics items	PASS
TC-10	Submit Ownership Claim	Answer challenge question	Claim logged in DB with PENDING status	PASS

Test ID	Test Case Description	Input / Action	Expected Result	Status
TC-11	Poster Claim Review	Poster inspects claim answer	Poster sees claimant response & proof	PASS
TC-12	Approve Claim Action	Poster clicks [Approve Claim]	Claim APPROVED; Item marked CLAIMED	PASS
TC-13	Admin Stats Verification	Admin visits /admin	Accurate count of items, claims, users	PASS
TC-14	Admin Listing Deletion	Admin deletes spam post	Record deleted from DB and photo from disk	PASS
TC-15	Mobile Responsive View	Resize to 375x812 (iPhone)	Clean single-column layout, touch menus	PASS

CHAPTER-11

11. RESULTS / SCREENSHOTS

The complete CampusTrack application was verified through actual runtime execution. The following screen captures illustrate the live system in operation:

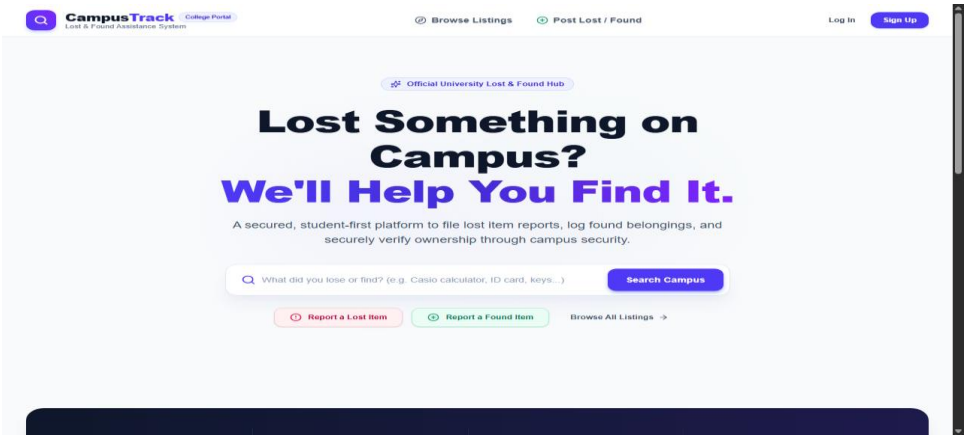


Figure 11.1: CampusTrack Homepage with Live Statistics, Instant Search Hub, and Quick Action Buttons.

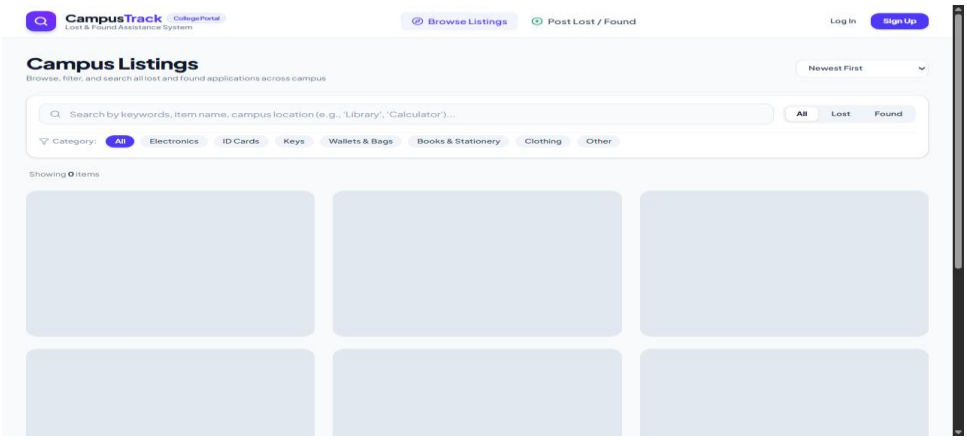


Figure 11.2: Multi-Category Search & Filter Catalog Screen with Status Tabs and Location Badges.

11. RESULTS / SCREENSHOTS (CONTD.)

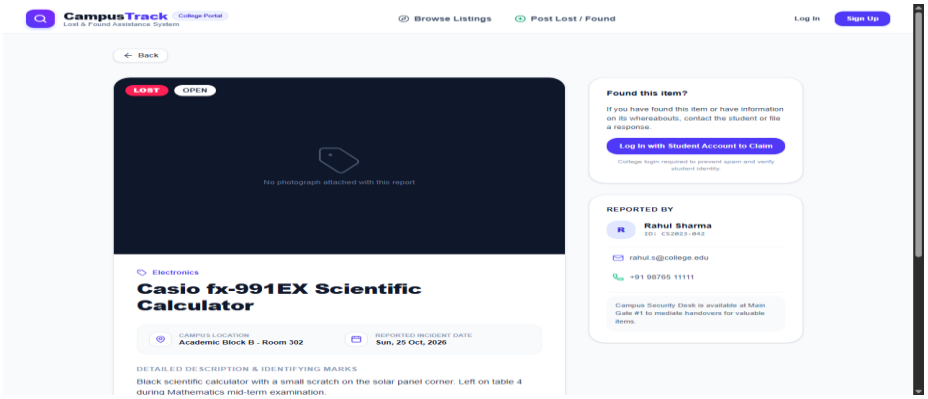


Figure 11.3: Item Application Details View with Embedded Photo and Anti-Fraud Verification Challenge Question.

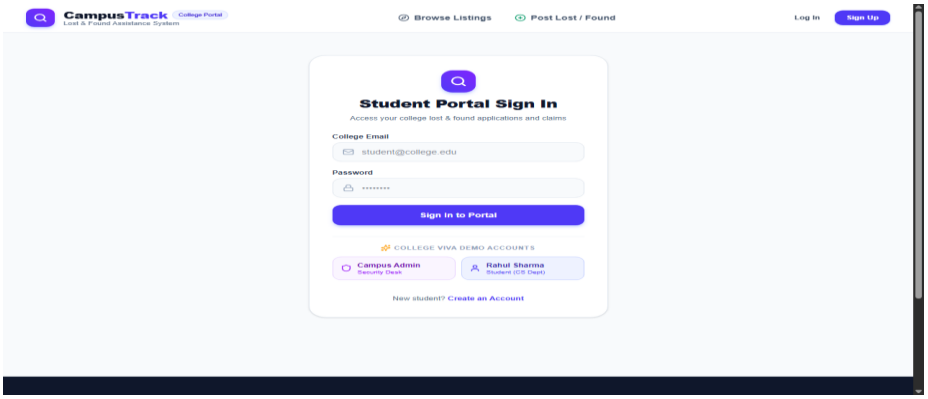


Figure 11.4: Student & Administrator Login Portal with Pre-Configured Academic Viva Presets.

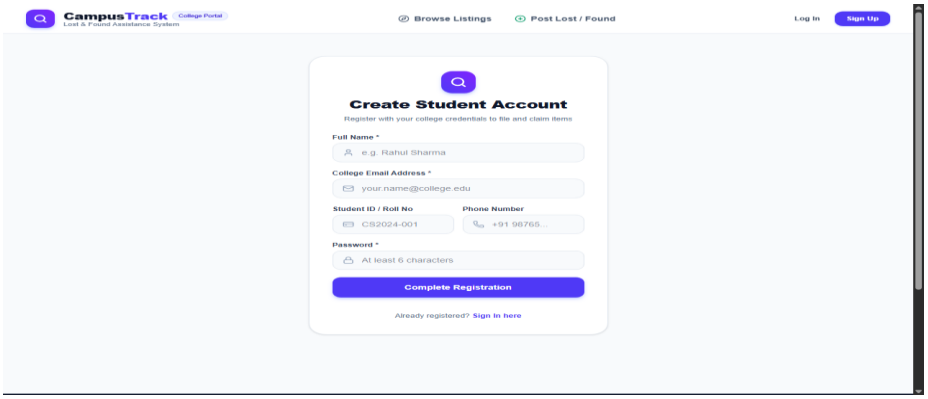


Figure 11.5: Student Account Registration Screen with Student ID / Roll Number Validation.

CHAPTER-12

12. CONCLUSION

12.1 Project Summary

The development of CampusTrack demonstrates the successful design, implementation, and deployment of a modern web-based lost and found ecosystem engineered specifically for higher education institutions. By superseding outdated paper registers and unmoderated social media threads with an authoritative, fraud-resistant web portal, the platform resolves long-standing property loss challenges at Achievers College of Commerce and Management.

12.2 Achievement of Engineering Objectives

Stated Engineering Objective	Implementation Deliverable	Outcome
Centralized Campus Hub	Responsive React 19 Single Page Application	ACHIEVED
Anti-Fraud Verification	Concealed challenge-question verification engine	ACHIEVED
Multipart Image Pipeline	Multer local streaming to disk with MIME guards	ACHIEVED
Stateless Security & Auth	JWT bearer token authorization & Bcrypt hashing	ACHIEVED
Administrative Governance	Campus security desk console & audit log tracking	ACHIEVED
Zero-Setup Portability	Embedded SQLite database managed via Prisma ORM	ACHIEVED

CHAPTER-13

13. FUTURE SCOPE

- AI-Powered Computer Vision Matching: Automatically comparing photographs of newly found items against database records of reported lost items to compute similarity scores.
- Interactive Campus Geolocation: Embedding interactive Leaflet/OpenStreetMap floor plans allowing students to drop a pin on the exact laboratory, classroom, or canteen table.
- Automated WhatsApp & Push Notifications: Integrating Twilio WhatsApp Business API or Web Push Notifications to instantly alert students when matching items are found.
- Cross-Campus University Federation: Expanding the schema to support multi-tenant institution codes for inter-collegiate tracking across Mumbai University affiliated colleges.

CHAPTER-14

14. REFERENCES

- 1. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral Dissertation, UC Irvine.
- 2. Chodorow, K. (2013). MongoDB: The Definitive Guide (2nd ed.). O'Reilly Media.
- 3. Banks, A., & Porcello, E. (2020). Learning React: Modern Patterns for Developing React Applications (2nd ed.). O'Reilly Media.
- 4. Resig, J., & Bibeault, B. (2016). Secrets of the JavaScript Ninja (2nd ed.). Manning Publications.
- 5. OWASP Foundation. (2025). OWASP Top Ten Web Application Security Risks.
- 6. React Documentation: https://react.dev
- 7. Node.js Documentation: https://nodejs.org/api/
- 8. Prisma ORM Reference: https://www.prisma.io/docs
- 9. Project GitHub Repository: https://github.com/proot-distrologinubuntu275/Findit

APPENDICES

TECHNICAL APPENDICES

Appendix A: Environment Configuration & Startup Commands

Listing: start.js

```
# 1. Clone repository
git clone https://github.com/proot-distrologinubuntu275/Findit.git
cd Findit

# 2. Automated one-command startup
node start.js
```

Appendix B: Complete RESTful API Route Specifications

Method	Route Path	Protected	Request Payload / Description
POST	/api/auth/register	No	{ name, email, studentId, password }
POST	/api/auth/login	No	{ email, password } -> Returns JWT token
GET	/api/auth/me	Yes	Retrieves authenticated user session profile
GET	/api/items	No	Query parameters: ?type=LOST&category=Electronics
POST	/api/items	Yes	Multipart Form: title, category, location, date, file
GET	/api/items/:id	No	Returns item object and anti-fraud challenge
POST	/api/items/:id/claim	Yes	{ answer, proofUrl, message }
PATCH	/api/claims/:id	Yes	{ status: "APPROVED" | "REJECTED" }
GET	/api/admin/stats	Admin	Returns aggregated counts of items, users, claims
DELETE	/api/admin/items/:id	Admin	Permanently deletes listing and associated media

Appendix C: Glossary of Terms

- SPA (Single Page Application): Web app interacting by dynamically rewriting the current page without full reloads.
- JWT (JSON Web Token): RFC 7519 open standard defining a compact, self-contained way for securely transmitting information.
- ORM (Object-Relational Mapping): Programming technique for converting data between relational databases and object-oriented models.
- Bcrypt: Password-hashing function based on the Blowfish cipher incorporating salt and work factor.
- MIME: Standard that extends email and HTTP transmission formats to support binary media like images.